

第四章

进程管理

主要内容

- 4.1 UNIX的进程调度状态
- 4.2 UNIX的进程切换调度**
- 4.3 UNIX的进程创建与终止
- 4.4 UNIX的进程图像交换
- 4.5 进程通信

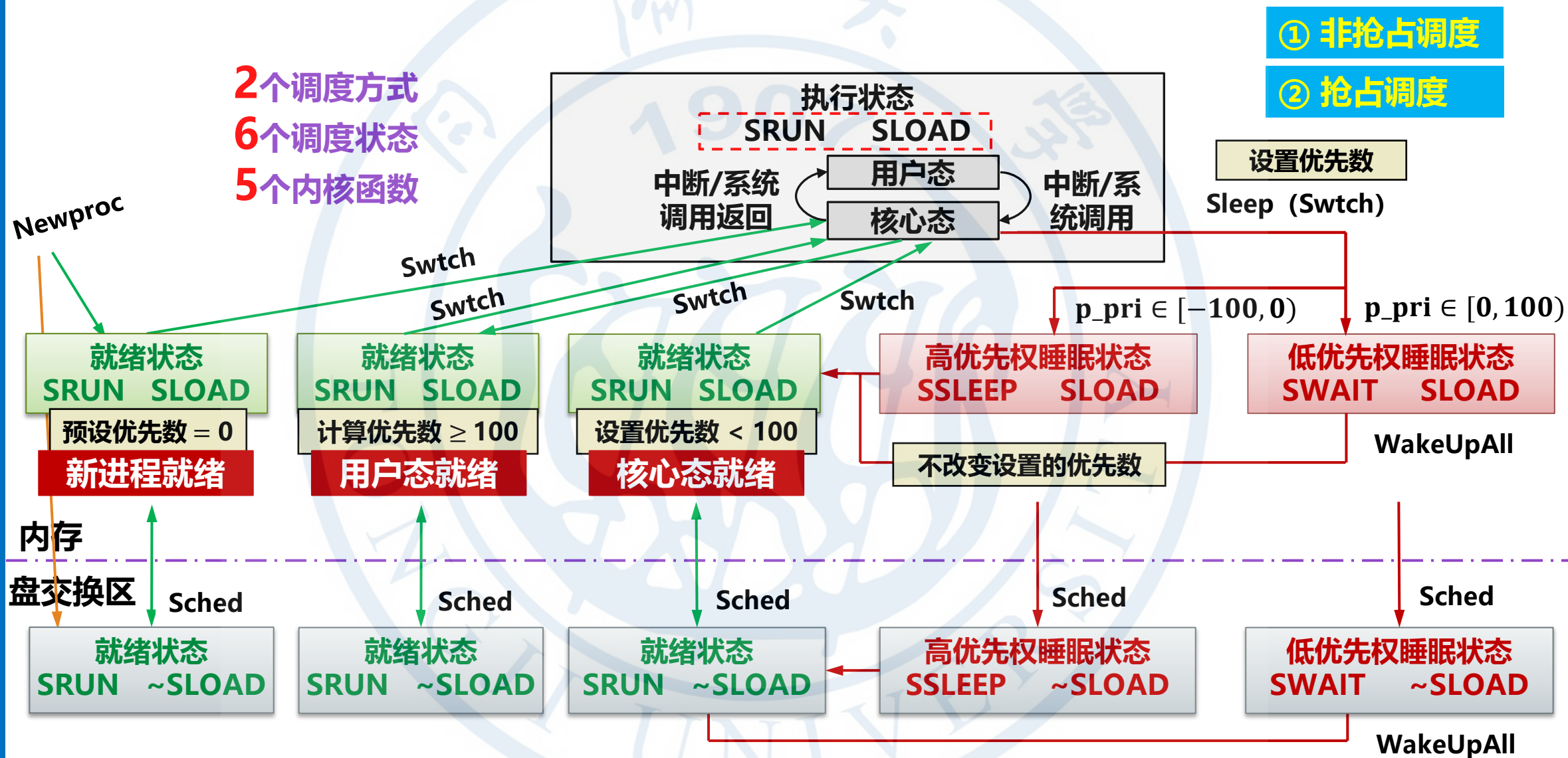


UNIX进程的调度状态



进程的调度状态

2个调度方式
6个调度状态
5个内核函数

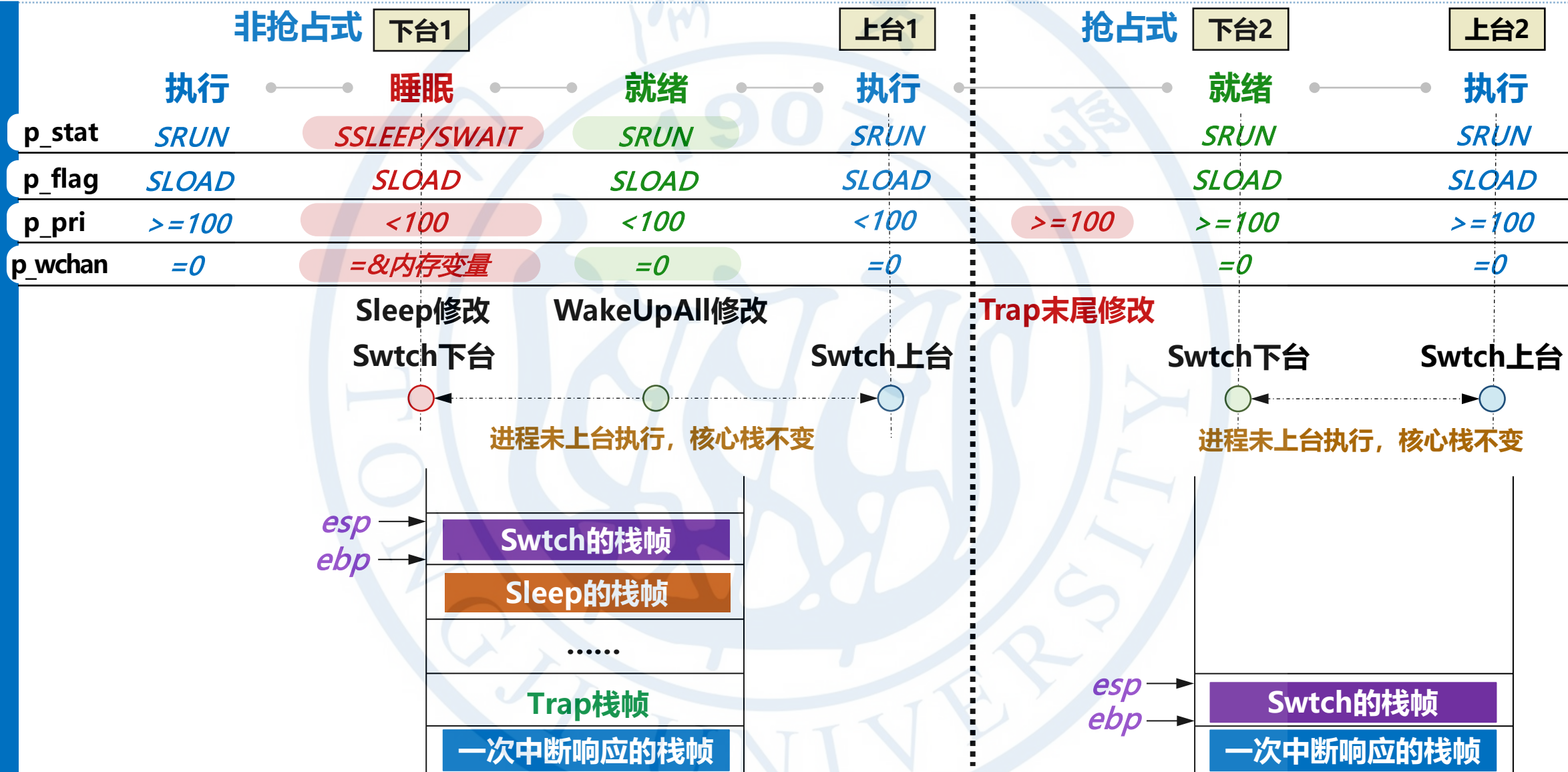




UNIX进程的调度状态



进程的调度状态





UNIX进程的调度状态



非抢占式

下台1

上台1

抢占式

下台2

上台2

	执行	睡眠	就绪	执行	就绪	执行
p_stat	SRUN	SSLEEP/SWAIT	SRUN	SRUN	SRUN	SRUN
p_flag	SLOAD	SLOAD	SLOAD	SLOAD	SLOAD	SLOAD
p_pri	≥ 100	< 100	< 100	< 100	≥ 100	≥ 100
p_wchan	=0	=&内存变量	=0	=0	=0	=0

Sleep修改
Swch下台

WakeUpAll修改

Swch上台

Trap末尾修改

Swch下台

Swch上台

先来看看进程Swch下台的原因....

进程的调度状态



UNIX进程的调度状态



非抢占式

下台1

上台1

抢占式

下台2

上台2

	执行	睡眠	就绪	执行	就绪	执行
p_stat	SRUN	SSLEEP/SWAIT	SRUN	SRUN	SRUN	SRUN
p_flag	SLOAD	SLOAD	SLOAD	SLOAD	SLOAD	SLOAD
p_pri	≥ 100	< 100	< 100	< 100	≥ 100	≥ 100
p_wchan	=0	=&内存变量	=0	=0	=0	=0

Sleep修改

WakeUpAll修改

Trap末尾修改

Swch下台

Swch上台

Swch下台

Swch上台

进程无法再执行下去，立即调用Swch放弃处理机

现运行进程入睡

现运行进程终止

进程的调度状态



UNIX进程的调度状态



进程的调度状态

	非抢占式 下台1			上台1	抢占式 下台2			上台2
	执行	睡眠	就绪	执行	就绪	就绪	就绪	执行
p_stat	SRUN	SSLEEP/SWAIT	SRUN	SRUN	SRUN	SRUN	SRUN	SRUN
p_flag	SLOAD	SLOAD	SLOAD	SLOAD	SLOAD	SLOAD	SLOAD	SLOAD
p_pri	≥ 100	< 100	< 100	< 100	≥ 100	≥ 100	≥ 100	≥ 100
p_wchan	=0	=&内存变量	=0	=0	=0	=0	=0	=0

Sleep修改

WakeUpAll修改

Swch下台

Swch上台

Trap末尾修改

Swch下台

Swch上台

进程无法再执行下去，立即调用Swch放弃处理机

现运行进程入睡

现运行进程终止

进程的睡眠与唤醒



UNIX进程的睡眠与唤醒



进程的睡眠

设置优先数:

```
static const int PSWP = -100;  
static const int PINOD = -90;  
static const int PRIBIO = -50;  
static const int EXPRI = -1;  
static const int PPIPE = 1;  
static const int TTIPRI = 10;  
static const int TTOPRI = 20;  
static const int PWAIT = 40;  
static const int PSLEP = 90;  
static const int PUSER = 100;
```

e.g.

Sleep(((unsigned long)bp), ProcessManager::PRIBIO);

睡眠原因
(一个内存地址)

优先数: -50

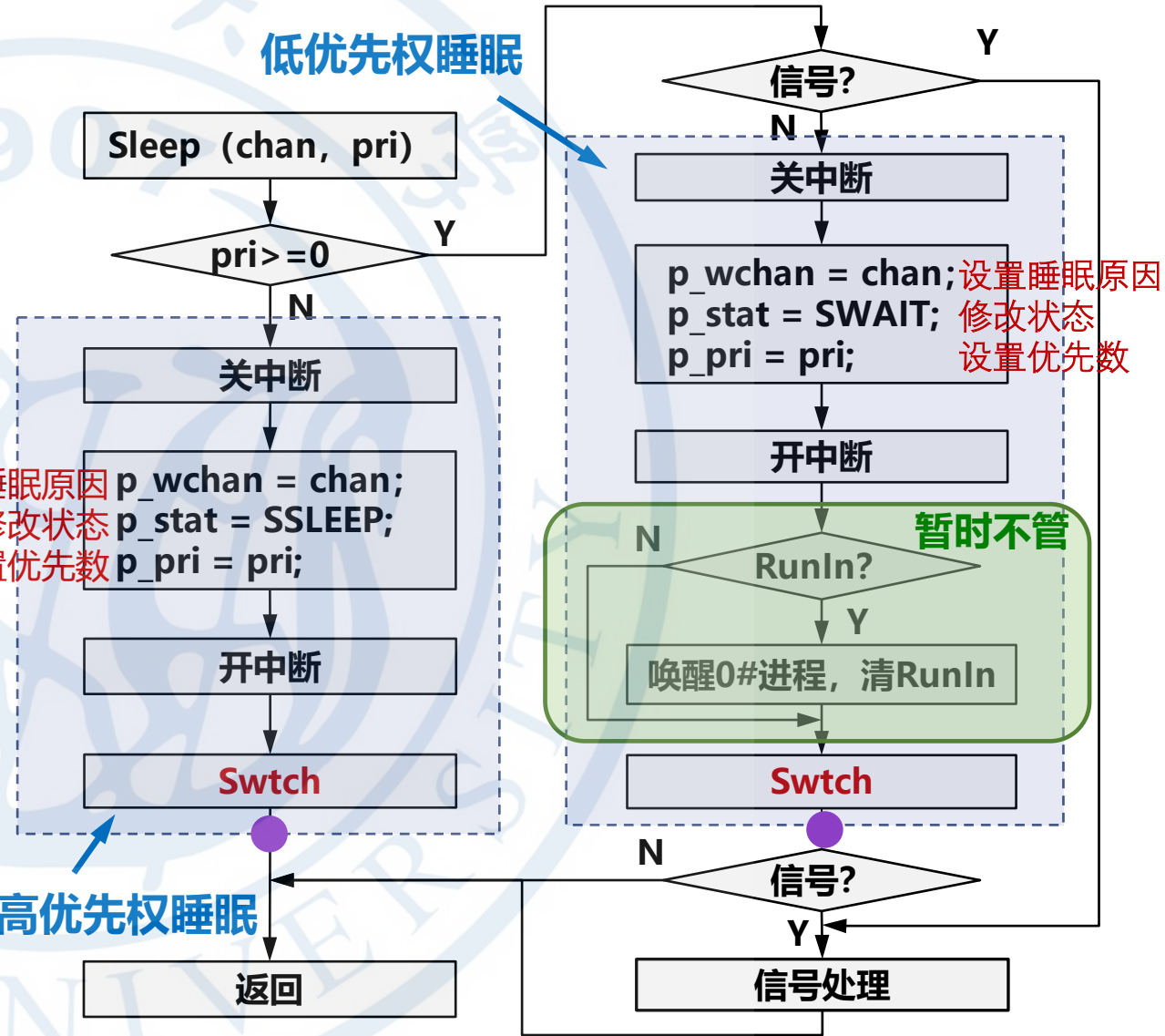
Swch的返回位置:

低优先权睡眠

设置睡眠原因
修改状态
设置优先数

高优先权睡眠

暂时不管

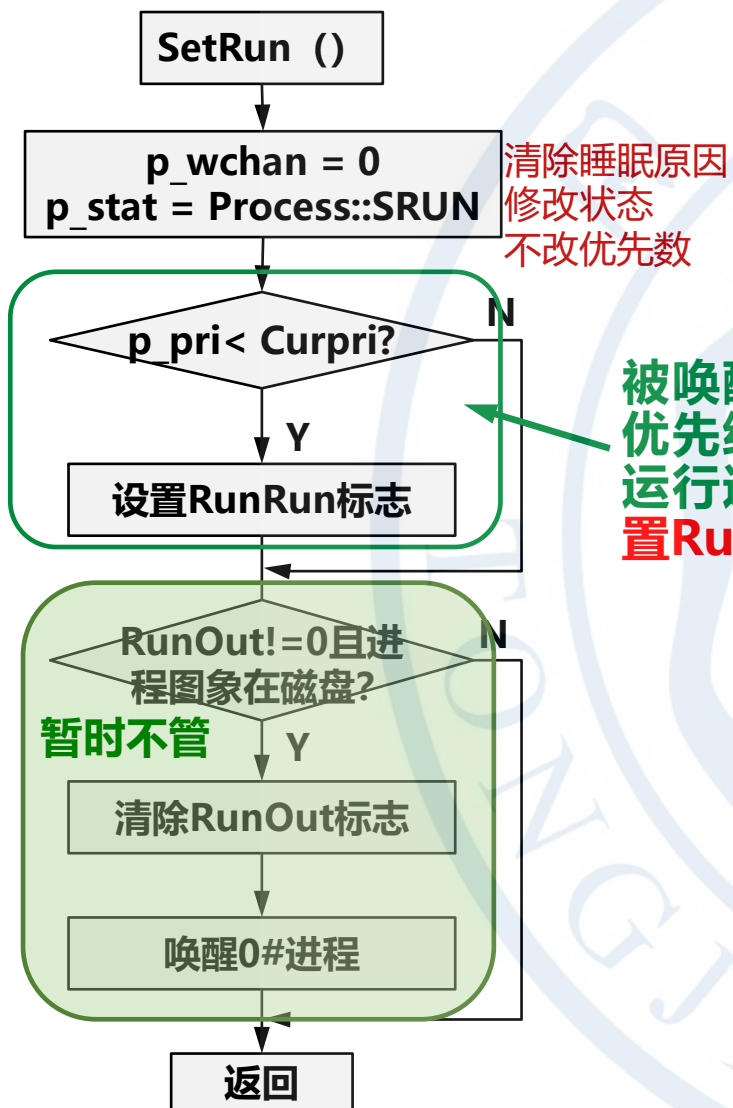




UNIX进程的睡眠与唤醒



进程的唤醒



```
void ProcessManager::WakeUpAll(unsigned long chan)
{
    for(int i = 0; i < ProcessManager::NPROC; i++)
    {
        if( this->process[i].IsSleepOn(chan) )
        {
            this->process[i].SetRun();
        }
    }
}
```

1. 以睡眠原因为参数
2. 核对所有的进程，看谁睡在这个原因上？
3. 调用SetRun将其唤醒

e.g.

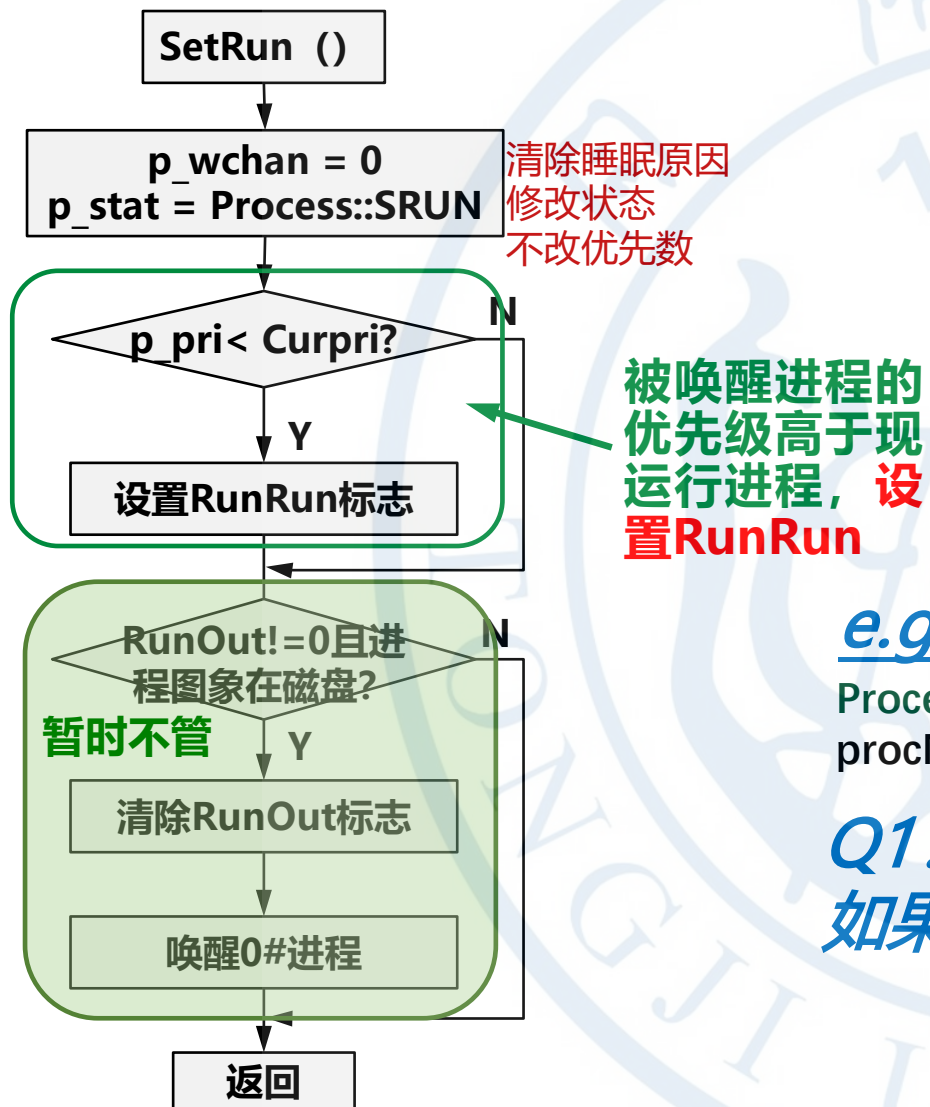
```
ProcessManager& procMgr = Kernel::Instance().GetProcessManager();
procMgr.WakeUpAll((unsigned long)bp);
```



UNIX进程的睡眠与唤醒



进程的唤醒



```
void ProcessManager::WakeUpAll(unsigned long chan)
{
    for(int i = 0; i < ProcessManager::NPROC; i++)
    {
        if( this->process[i].IsSleepOn(chan) )
        {
            this->process[i].SetRun();
        }
    }
}
```

1. 以睡眠原因为参数
2. 核对所有的进程，看谁睡在这个原因上？
3. 调用SetRun将其唤醒

e.g.

```
ProcessManager& procMgr = Kernel::Instance().GetProcessManager();
procMgr.WakeUpAll((unsigned long)bp);
```

Q1:

如果睡在chan这个原因上的进程不止一个呢？



UNIX进程的睡眠与唤醒



e.g. UNIX系统中提供了一个定时睡眠的系统调用

```
main()
{
    .....;
    sleep(2); /* 睡两秒后再执行 */
    .....;
}
```

Time::tout: 闹钟
设置为距离当前时间最近的需要唤醒进程的时间点

Time::time
当前系统时间



UNIX进程的睡眠与唤醒



e.g. UNIX系统中提供了一个定时睡眠的系统调用

```
main()
{
    .....;
    sleep(2); /* 睡两秒后再执行 */
    .....;
}
```

```
int SystemCall::Sys_Ssleep()
{
    User& u = Kernel::Instance().GetUser();
    X86Assembly::CLI();

    进程需要被唤醒的时间 = 当前的系统时间 + 进程需要入睡的时间
    unsigned int wakeTime = Time::time + u.u_arg[0]; /* sleep(second) */

    while( wakeTime > Time::time ) /* 现运行进程还没到该醒的时候 */
    {
        if ( Time::tout <= Time::time || Time::tout > wakeTime )
            /* “没有闹钟” 或者 “闹钟时间晚于现运行进程需要被叫醒的时间” */
        {
            Time::tout = wakeTime; /* 将闹钟设置为现运行进程需要被唤醒的时间 */
        }
        /* 现运行进程入睡，原因为Time::tout的地址，优先级为90 */
        u.u_procp->Sleep((unsigned long)&Time::tout, ProcessManager::PSLEP);
    }
    X86Assembly::STI();
    return 0;
}
```




UNIX进程的睡眠与唤醒



进程的唤醒

e.g. UNIX系统中提供了一个定时睡眠的系统调用

```
main()
{
    .....;
    sleep(2); /* 睡两秒后再执行 */
    .....;
}
```

```
int SystemCall::Sys_Ssleep()
```

```
{
    User& u = Kernel::Instance().GetUser();
    X86Assembly::CLI();
```

进程需要被唤醒的时间 = 当前的系统时间 + 进程需要入睡的时间

```
unsigned int wakeTime = Time::time + u.u_arg[0]; /* sleep(second) */
```

```
while( wakeTime > Time::time ) /* 现运行进程还没到该醒的时候 */
```

```
{
    if ( Time::tout <= Time::time || Time::tout > wakeTime )
        /* “没有闹钟” 或者 “闹钟时间晚于现运行进程需要被叫醒的时间” */
```

```
{
    Time::tout = wakeTime; /* 将闹钟设置为现运行进程需要被唤醒的时间 */
```

```
}
/* 现运行进程入睡，原因为Time::tout的地址，优先级为90 */
```

```
u.u_procp->Sleep((unsigned long)&Time::tout, ProcessManager::PSLEP);
```

```
}
```

整数秒的时钟中断里：

```
if ( Time::time == Time::tout ) /* 如果闹钟时间到 */
```

```
{
    procMgr.WakeUpAll((unsigned long)&Time::tout); /* 唤醒所有睡在Time::tout的地址上的进程 */
}
```



UNIX进程的睡眠与唤醒



进程的唤醒

e.g. UNIX系统中提供了一个定时睡眠的系统调用

```
main()
{
    .....;
    sleep(2); /* 睡两秒后再执行 */
    .....;
}
```

1. 该醒的进程 `wakeTime == Time::time`: 上台后退出while循环, 从Sys_Ssleep返回;

```
int SystemCall::Sys_Ssleep()
```

```
{
    User& u = Kernel::Instance().GetUser();
    X86Assembly::CLI();
```

进程需要被唤醒的时间 = 当前的系统时间 + 进程需要入睡的时间

```
unsigned int wakeTime = Time::time + u.u_arg[0]; /* sleep(second) */
```

```
while( wakeTime > Time::time ) /* 现运行进程还没到该醒的时候 */
{
```

```
    if ( Time::tout <= Time::time || Time::tout > wakeTime )
        /* “没有闹钟” 或者 “闹钟时间晚于现运行进程需要被叫醒的时间” */
```

```
    {
        Time::tout = wakeTime; /* 将闹钟设置为现运行进程需要被唤醒的时间 */
    }
```

```
    /* 现运行进程入睡, 原因为Time::tout的地址, 优先级为90 */
```

```
    u.u_procp->Sleep((unsigned long)&Time::tout, ProcessManager::PSLEP);
```

```
}
```

整数秒的时钟中断里:

```
if ( Time::time == Time::tout ) /* 如果闹钟时间到 */
```

```
{
    procMgr.WakeUpAll((unsigned long)&Time::tout); /* 唤醒所有睡在Time::tout的地址上的进程 */
}
```



UNIX进程的睡眠与唤醒



进程的唤醒

e.g. UNIX系统中提供了一个定时睡眠的系统调用

```
main()
{
    .....;
    sleep(2); /* 睡两秒后再执行 */
    .....;
}
```

1. 该醒的进程 `wakeTime == Time::time`: 上台后退出while循环, 从`Sys_Sslep`返回;

2. 不该醒的进程 `wakeTime > Time::time`: 上台后, 判断是否需要修改闹钟, 然后再睡。

整数秒的时钟中断里:

```
if ( Time::time == Time::tout ) /* 如果闹钟时间到 */
{
    procMgr.WakeUpAll((unsigned long)&Time::tout); /* 唤醒所有睡在Time::tout的地址上的进程 */
}
```

```
int SystemCall::Sys_Sslep()
{
```

```
    User& u = Kernel::Instance().GetUser();
    X86Assembly::CLI();
```

进程需要被唤醒的时间 = 当前的系统时间 + 进程需要入睡的时间

```
    unsigned int wakeTime = Time::time + u.u_arg[0]; /* sleep(second) */
```

```
    while( wakeTime > Time::time ) /* 现运行进程还没到该醒的时候 */
    {
```

```
        if ( Time::tout <= Time::time || Time::tout > wakeTime )
            /* “没有闹钟” 或者 “闹钟时间晚于现运行进程需要被叫醒的时间” */
```

```
        {
            Time::tout = wakeTime; /* 将闹钟设置为现运行进程需要被唤醒的时间 */
        }
```

```
        /* 现运行进程入睡, 原因为Time::tout的地址, 优先级为90 */
```

```
        u.u_procp->Sleep((unsigned long)&Time::tout, ProcessManager::PSLEP);
```

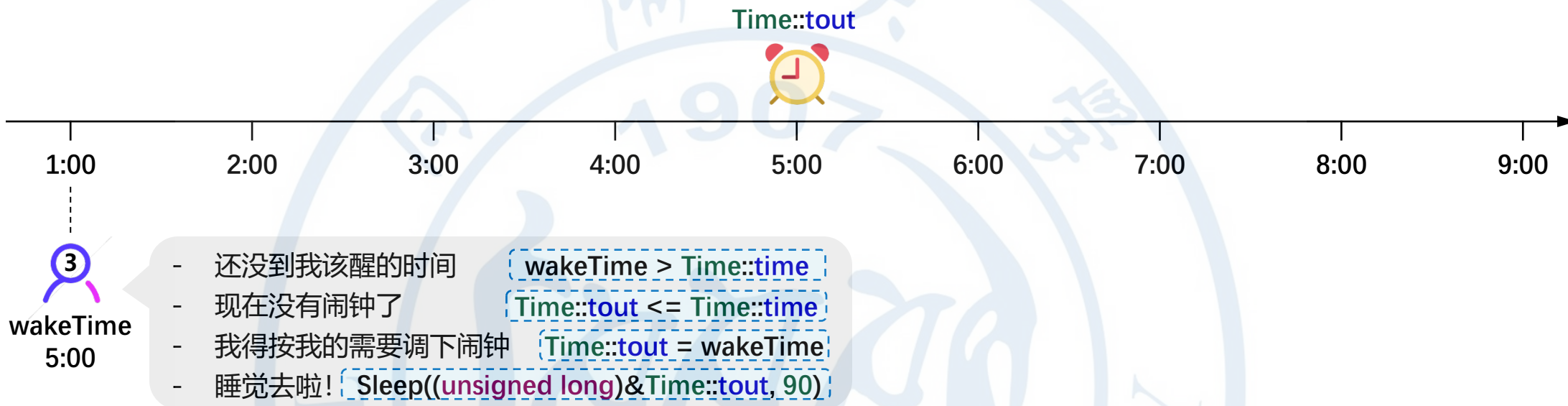
```
    }
```




UNIX进程的睡眠与唤醒



进程的唤醒

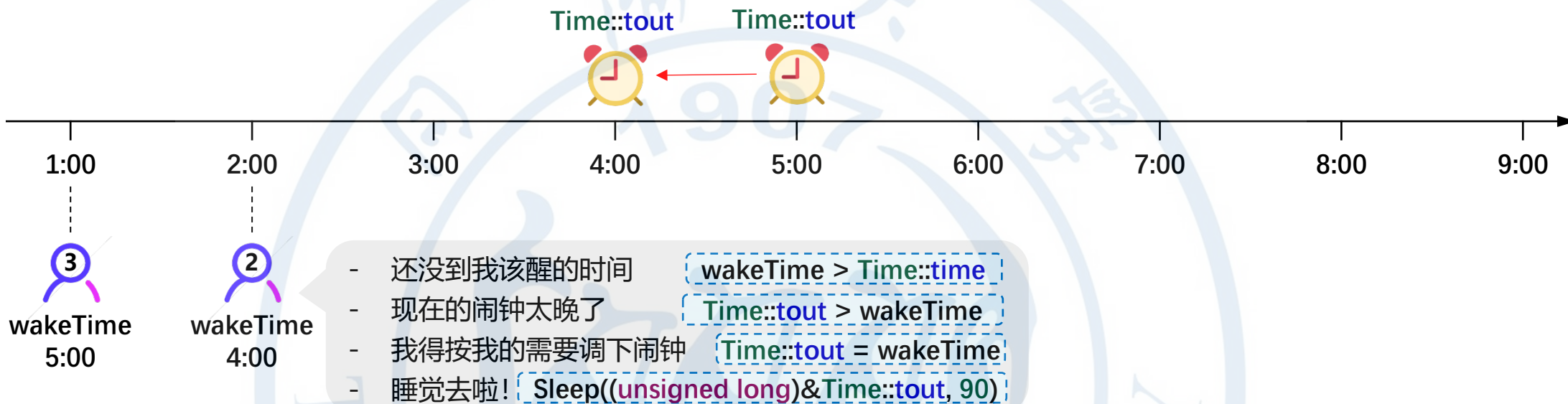




UNIX进程的睡眠与唤醒



进程的唤醒

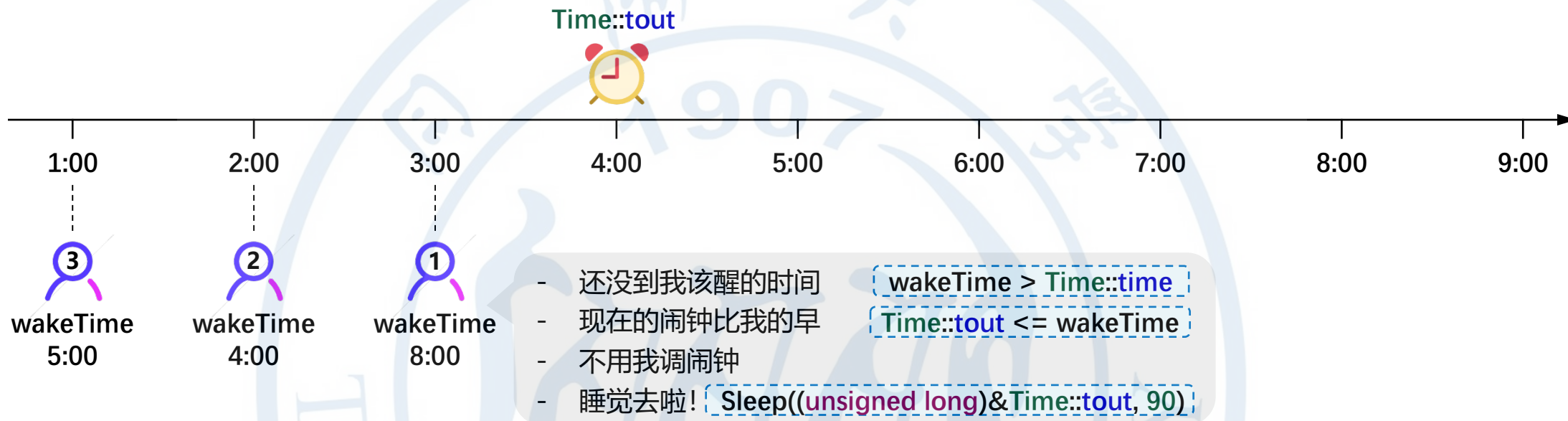




UNIX进程的睡眠与唤醒



进程的唤醒

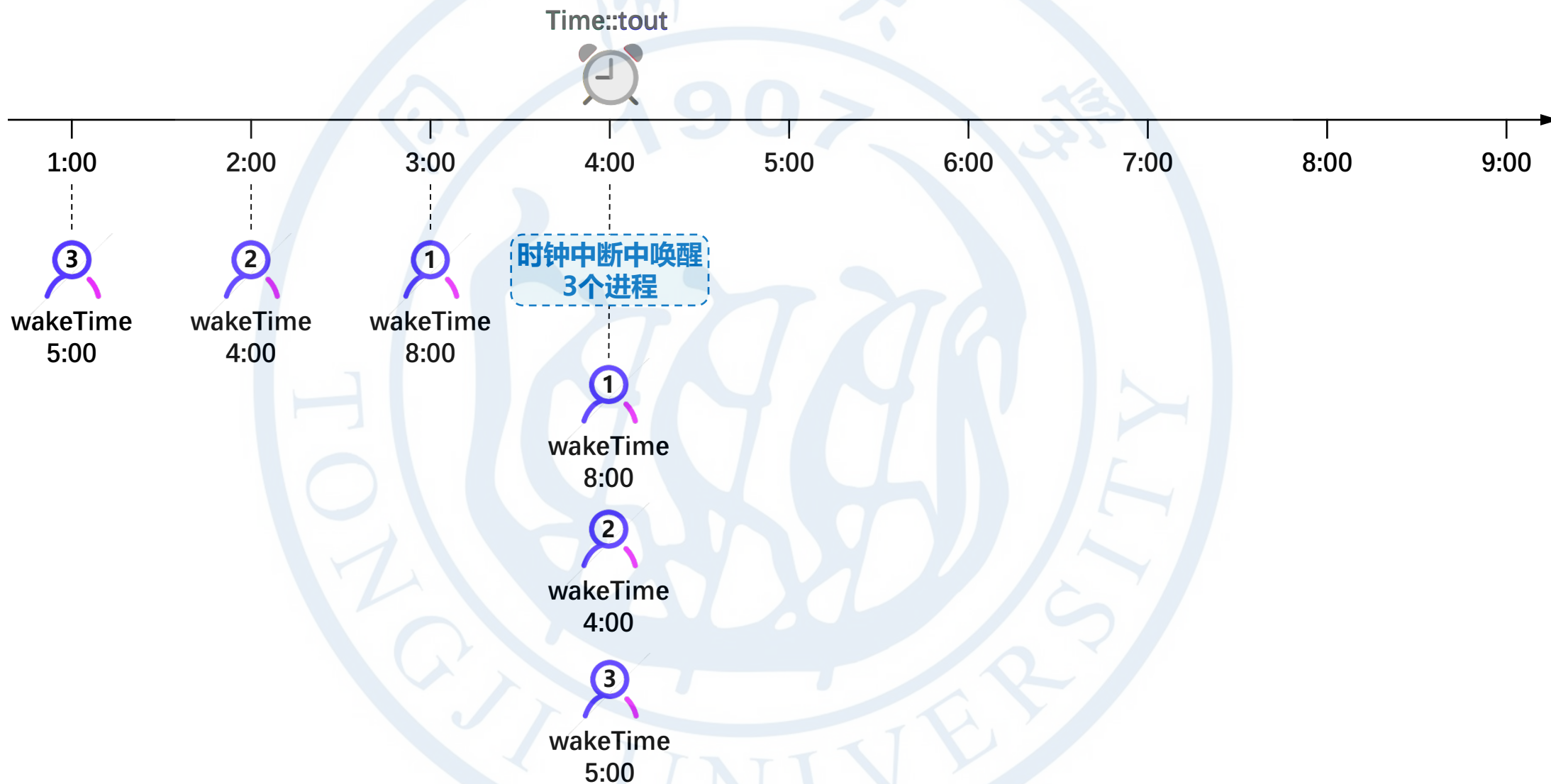




UNIX进程的睡眠与唤醒



进程的唤醒

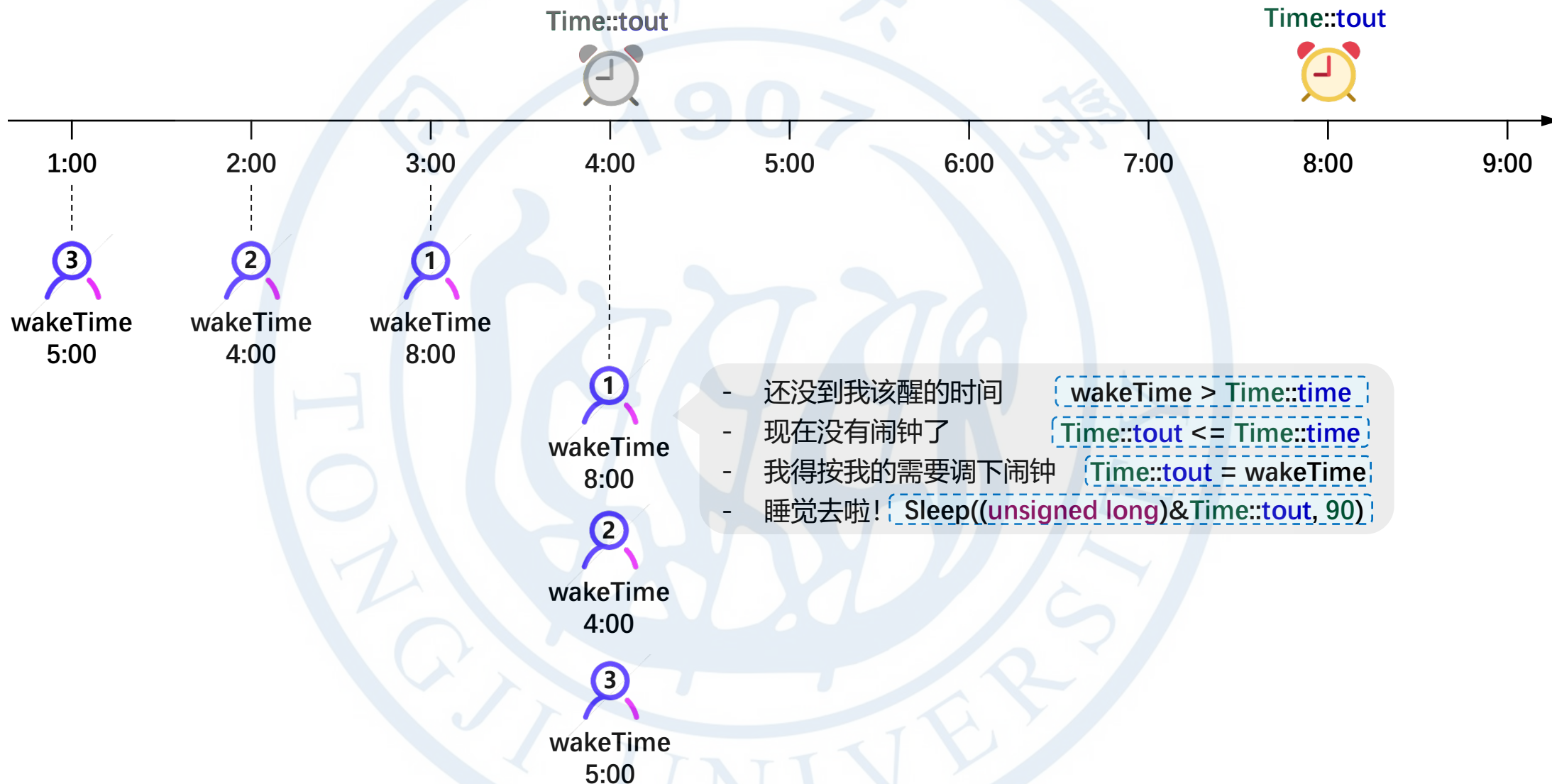




UNIX进程的睡眠与唤醒



进程的唤醒

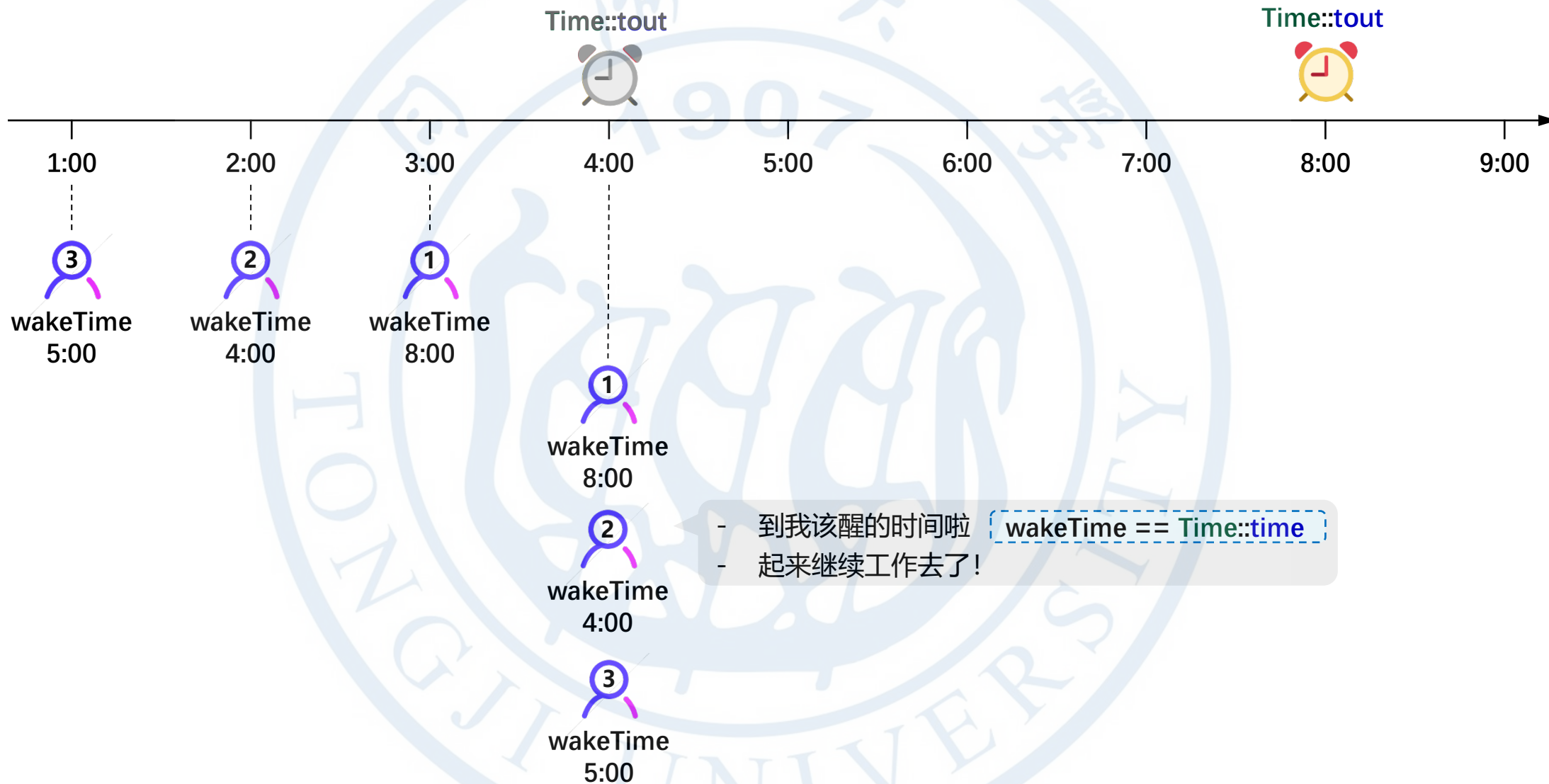




UNIX进程的睡眠与唤醒



进程的唤醒

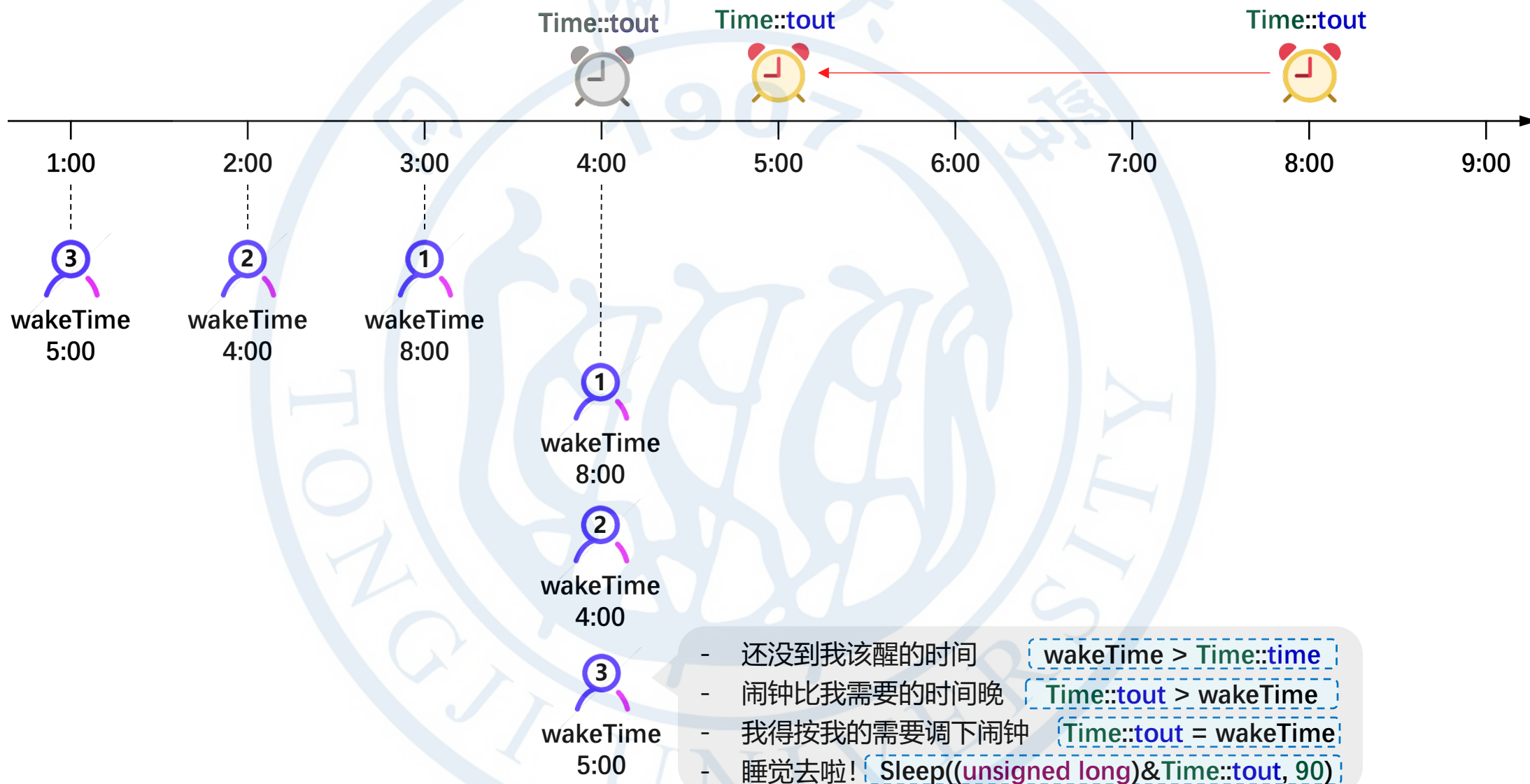




UNIX进程的睡眠与唤醒



进程的唤醒

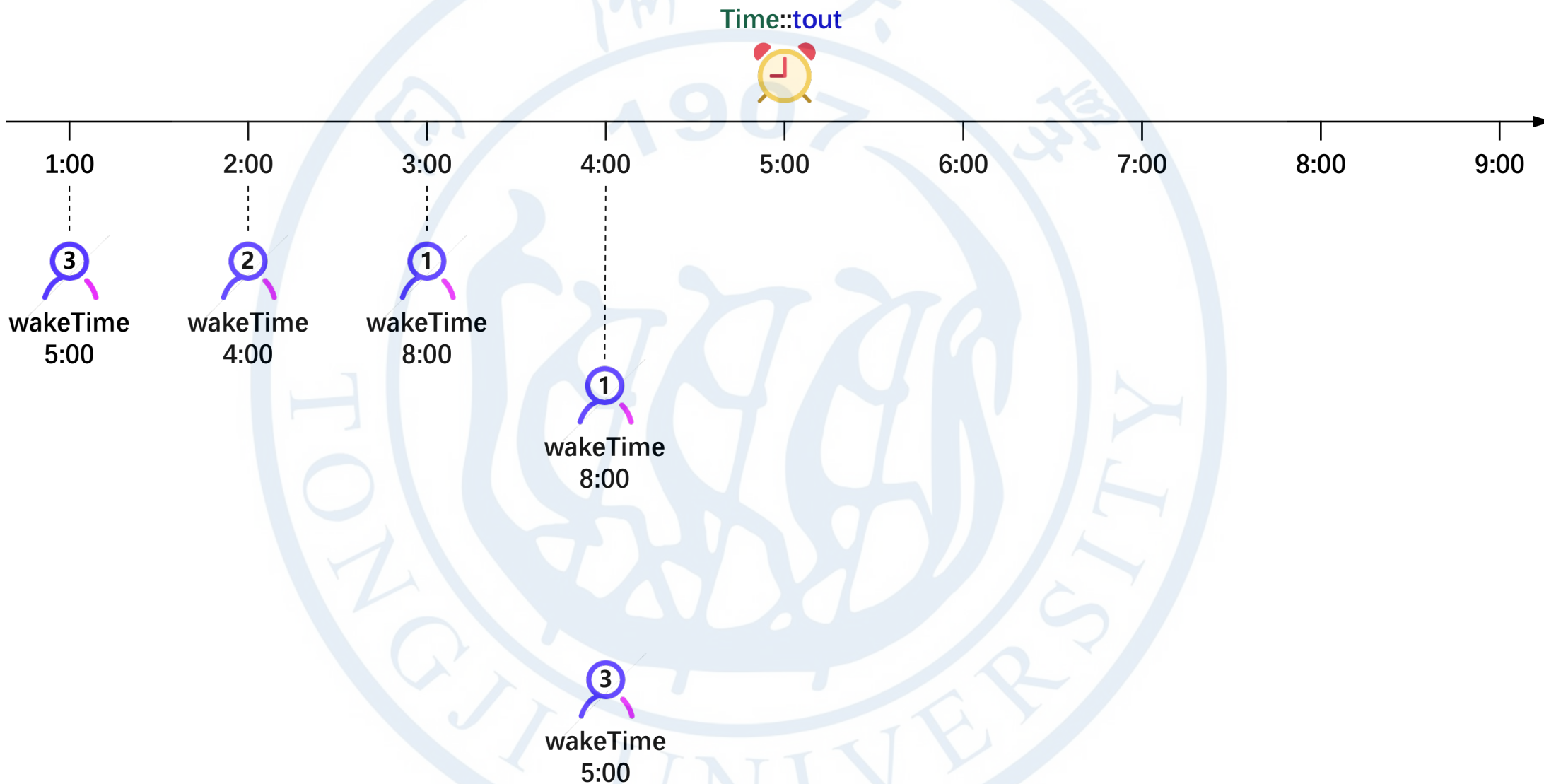




UNIX进程的睡眠与唤醒



进程的唤醒





UNIX进程的睡眠与唤醒



进程的唤醒





UNIX进程的睡眠与唤醒



e.g. UNIX系统中提供了一个定时睡眠的系统调用

```
main()
{
    .....;
    sleep(2); /* 睡两秒后再执行 */
    .....;
}
```

```
int SystemCall::Sys_Ssleep()
{
    User& u = Kernel::Instance().GetUser();
    X86Assembly::CLI(); // 开关中断的目的?

    进程需要被唤醒的时间 = 当前的系统时间 + 进程需要入睡的时间

    unsigned int wakeTime = Time::time + u.u_arg[0]; /* sleep(second) */

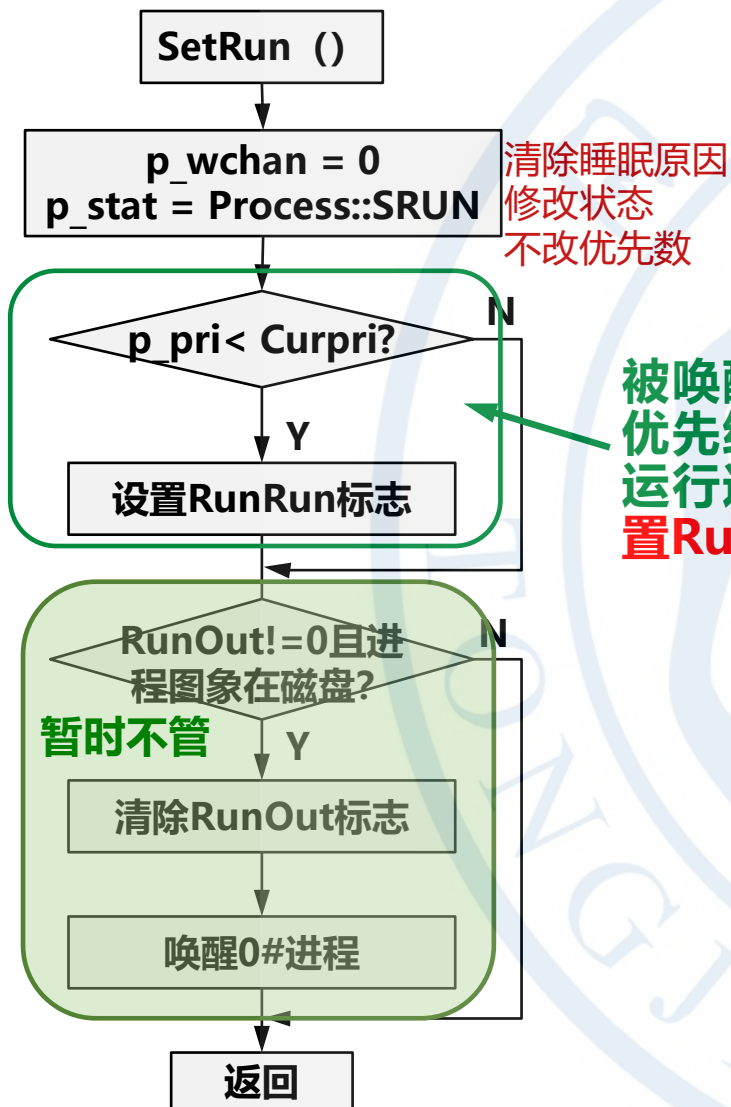
    while( wakeTime > Time::time ) /* 现运行进程还没到该醒的时候 */
    {
        if ( Time::tout <= Time::time || Time::tout > wakeTime )
            /* “没有闹钟” 或者 “闹钟时间晚于现运行进程需要被叫醒的时间” */
        {
            Time::tout = wakeTime; /* 将闹钟设置为现运行进程需要被唤醒的时间 */
        }
        /* 现运行进程入睡, 原因为Time::tout的地址, 优先级为90 */
        u.u_procp->Sleep((unsigned long)&Time::tout, ProcessManager::PSLEP);
    }
    X86Assembly::STI();
    return 0;
}
```



UNIX进程的睡眠与唤醒



进程的唤醒



```
void ProcessManager::WakeUpAll(unsigned long chan)
{
    for(int i = 0; i < ProcessManager::NPROC; i++)
    {
        if( this->process[i].IsSleepOn(chan) )
        {
            this->process[i].SetRun();
        }
    }
}
```

1. 以睡眠原因为参数
2. 核对所有的进程，看谁睡在这个原因上？
3. 调用SetRun将其唤醒

e.g.

```
ProcessManager& procMgr = Kernel::Instance().GetProcessManager();
procMgr.WakeUpAll((unsigned long)bp);
```

Q1:

如果睡在chan这个原因上的进程不止一个呢？

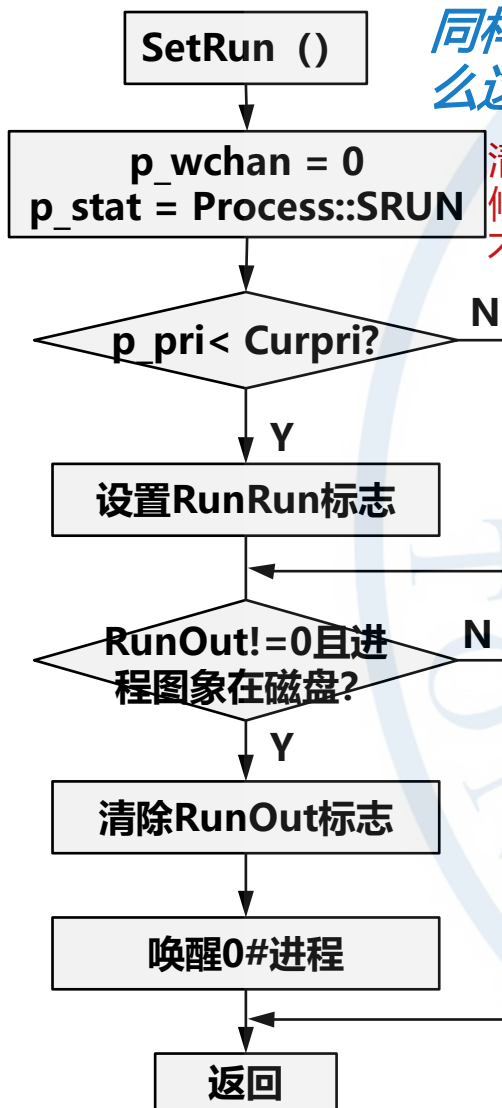
A1: 全叫起来，不该醒的继续睡。



UNIX进程的睡眠与唤醒

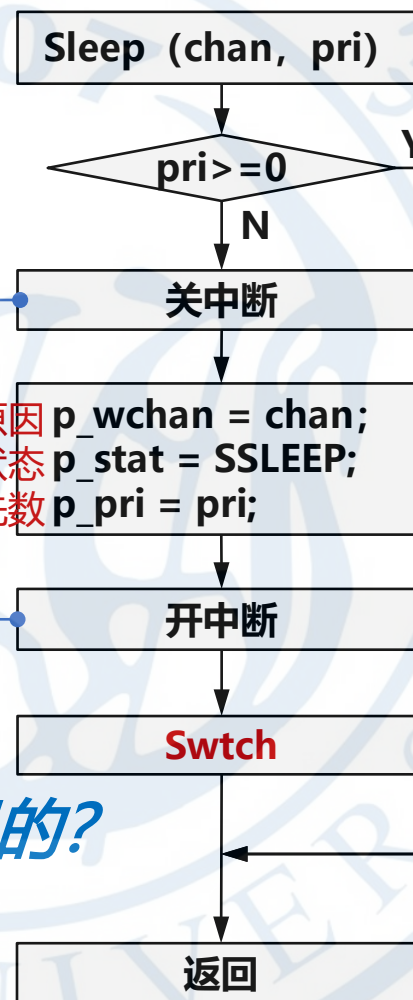


进程的唤醒



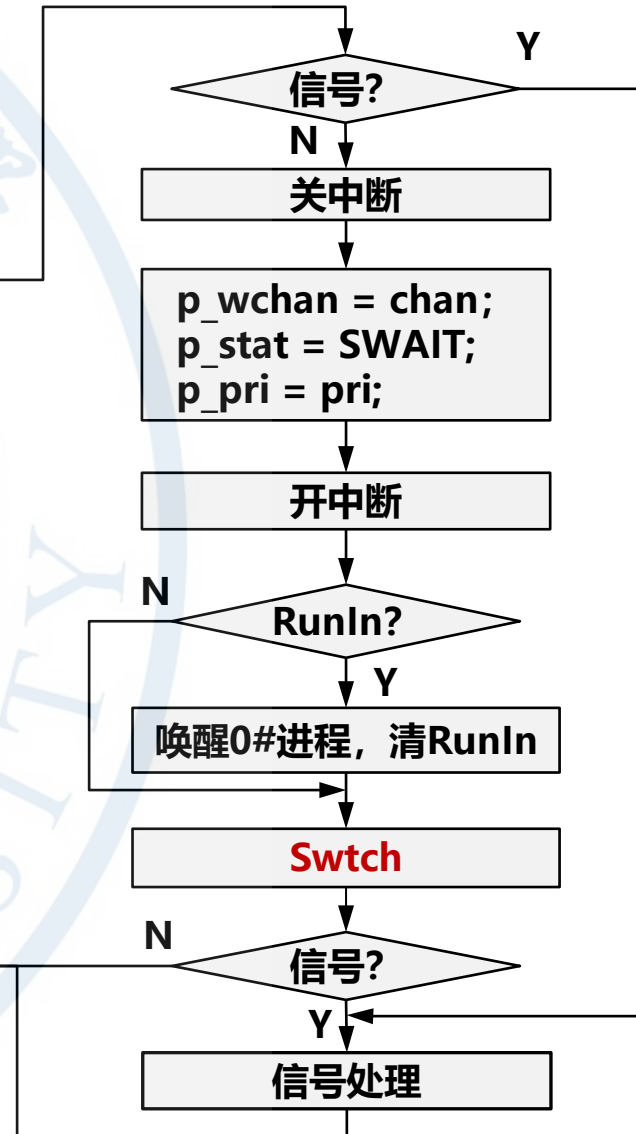
同样修改这几个参数，为什么这里不需要开关中断？

清除睡眠原因
修改状态
不改优先数



设置睡眠原因
修改状态
设置优先数

Q2:
这里开关中断的目的？

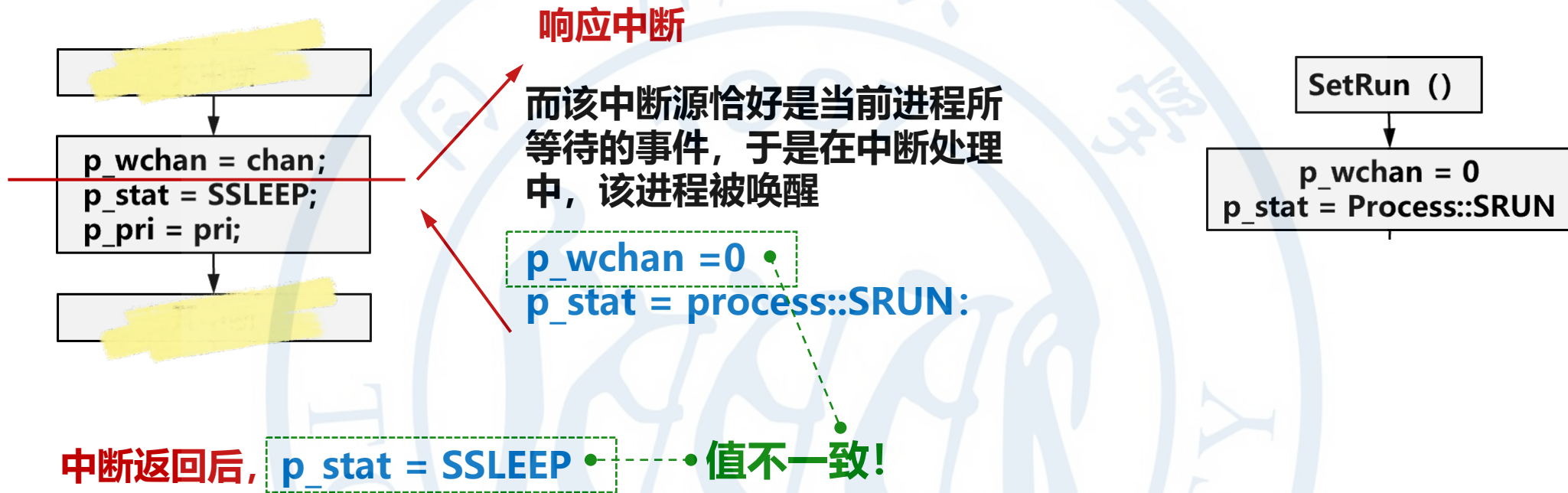




UNIX进程的睡眠与唤醒



进程的唤醒



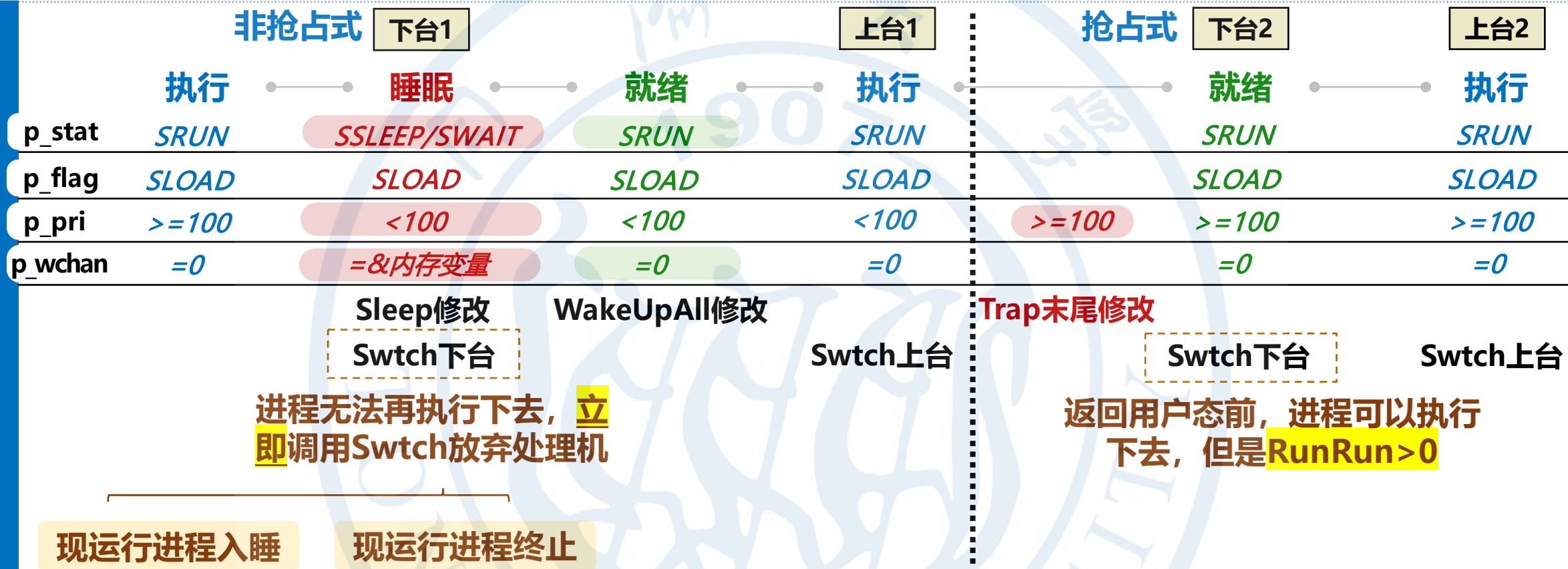
WakeUpAll程序中永远不可能唤醒了!



UNIX进程的调度状态



进程的调度状态

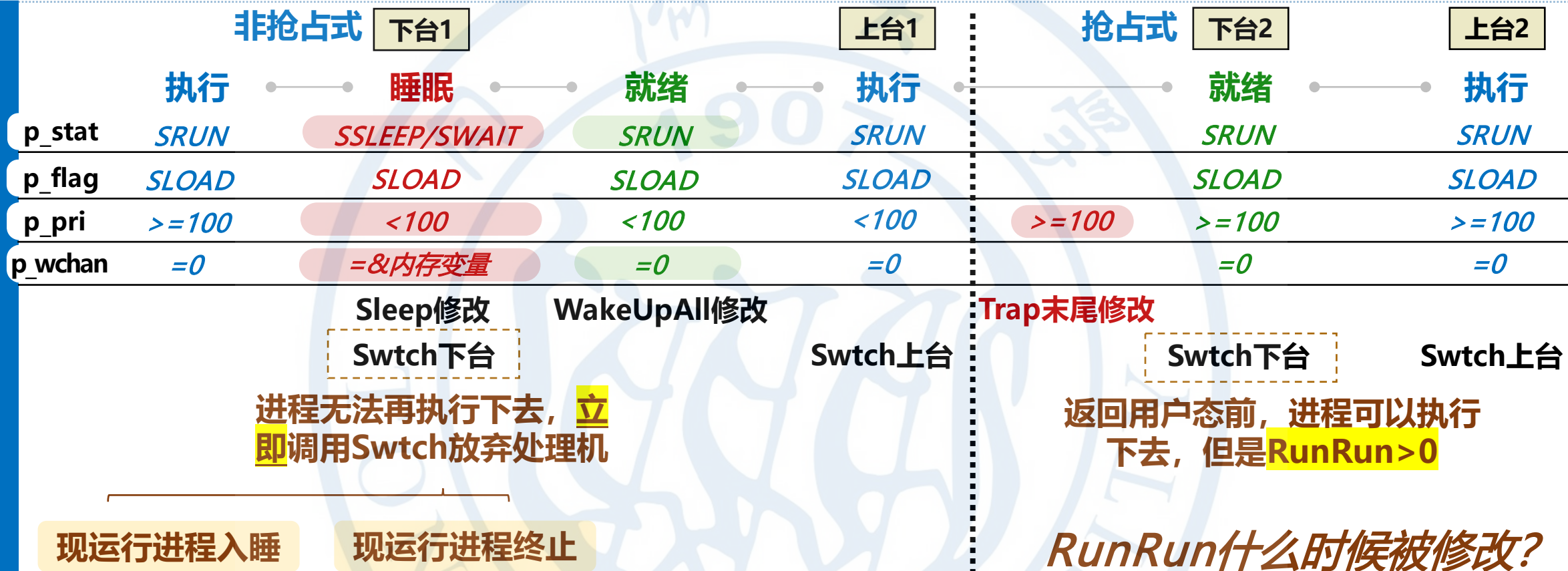




UNIX进程的调度状态



进程的调度状态





UNIX进程的调度状态



1. 重算优先数时，对RunRun的修改：



1. 整数秒，重算所有用户态就绪的进程的优先数



2. 整数秒，重算现运行进程的优先数



3. 系统调用末尾，重算现运行进程的优先数

Setpri函数完成进程优先数的计算

```
void Process::SetPri()
{
    int priority;
    ProcessManager& procMgr = Kernel::Instance().GetProcessManager();
    priority = this->p_cpu / 16;
    priority += ProcessManager::PUSER + this->p_nice;
    if ( priority > 255 )
    {
        priority = 255;
    }
    if ( priority > procMgr.CurPri )
    {
        procMgr.RunRun++;
    }
    this->p_pri = priority;
}
```

计算进程优先数

如果算得的优先数 > 现运行进程上台时的优先数

RunRun标志位被设置



重算优先数



UNIX进程的调度状态



非抢占式

下台1

上台1

抢占式

下台2

上台2

	执行	睡眠	就绪	执行	就绪	执行
p_stat	SRUN	SSLEEP/SWAIT	SRUN	SRUN	SRUN	SRUN
p_flag	SLOAD	SLOAD	SLOAD	SLOAD	SLOAD	SLOAD
p_pri	≥ 100	< 100	< 100	< 100	≥ 100	≥ 100
p_wchan	=0	=&内存变量	=0	=0	=0	=0



3. 系统调用末尾，重算现运行进程的优先数

Trap末尾修改

Setpri有两个目的：

1. 刷掉核心态下的优先级，恢复计算获得的优先数
2. 现运行进程优先级下降，设置RunRun

重算优先数



UNIX进程的调度状态



重算优先数



1. 整数秒，重算**所有** p_pri
> **PUSER**进程的优先数

*既重算现运行进程，也重算
其他所有的用户态就绪进程*



UNIX进程的调度状态



重算优先数

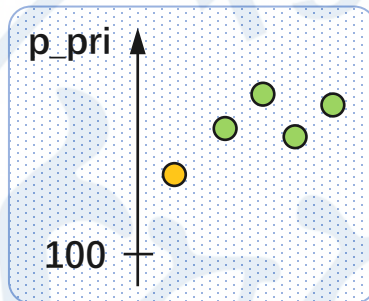


1. 整数秒，重算所有 p_pri
> $PUSER$ 进程的优先数

既重算现运行进程，也重算
其他所有的用户态就绪进程

这些进程的优先数：

1. 在 t 时刻时钟中断中获得
计算优先数（此时，
 p_pri 体现截至到 t 时刻
的 p_cpu ）；





UNIX进程的调度状态



重算优先数



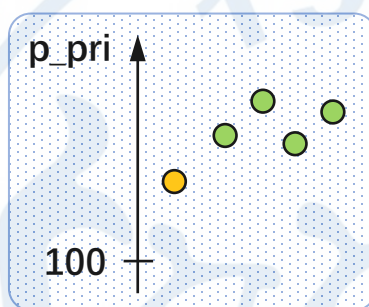
1. 整数秒，重算所有 $p_pri > PUSER$ 进程的优先数

既重算现运行进程，也重算其他所有的用户态就绪进程

这些进程的优先数：

1. 在 t 时刻时钟中断中获得计算优先数（此时， p_pri 体现截至到 t 时刻的 p_cpu ）；

2. 在 $t' \in [t, t + \Delta t]$ 时刻内曾在核心态下执行 Trap 末尾重算（此时， p_pri 体现截至到 t' 的 p_cpu ）





UNIX进程的调度状态



重算优先数



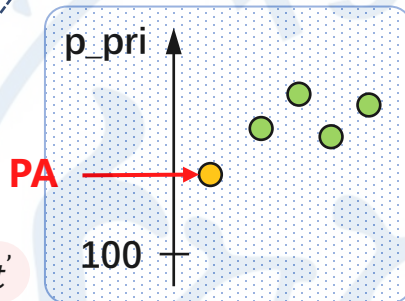
1. 整数秒, 重算所有 $p_pri > PUSER$ 进程的优先数

既重算现运行进程, 也重算其他所有的用户态就绪进程

这些进程的优先数:

1. 在 t 时刻时钟中断中获得计算优先数 (此时, p_pri 体现截至到 t 时刻的 p_cpu) ;

2. 在 $t' \in [t, t + \Delta t]$ 时刻内曾在核心态下执行 Trap 末尾重算 (此时, p_pri 体现截至到 t' 的 p_cpu)



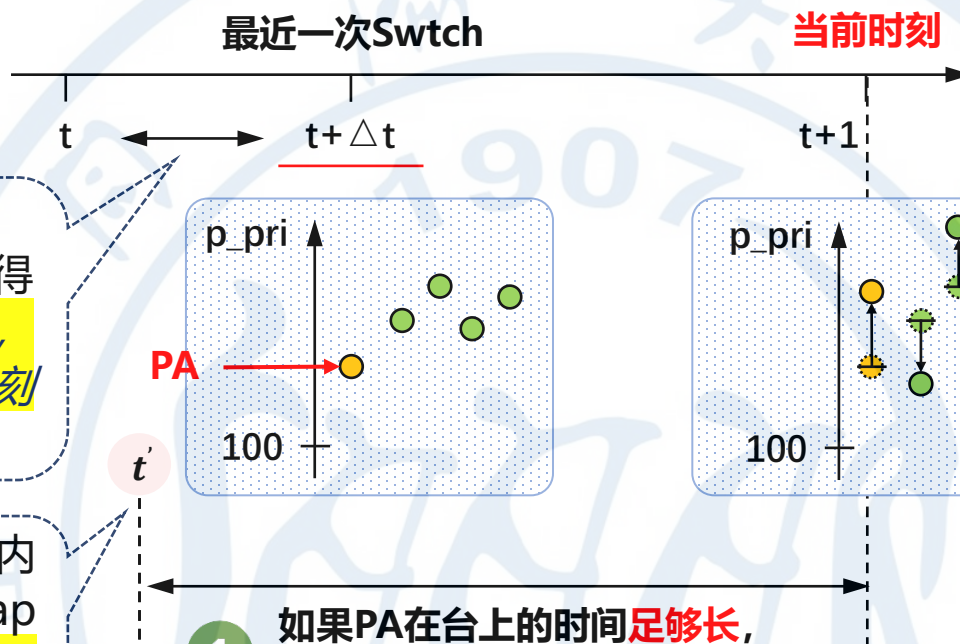
最近一次Swch中, p_pri 最小的进程 PA (截止到 $t + \Delta t$, p_pri 最小, 优先级最高的进程) 抢到 CPU



UNIX进程的调度状态



重算优先数



这些进程的优先数:

1. 在 t 时刻时钟中断中获得计算优先数 (此时, p_pri 体现截至到 t 时刻的 p_cpu) ;

2. 在 $t' \in [t, t + \Delta t]$ 时刻内曾在核心态下执行 Trap 末尾重算 (此时, p_pri 体现截至到 t' 的 p_cpu)

1. 整数秒, 重算所有 $p_pri > PUSER$ 进程的优先数

既重算现运行进程, 也重算其他所有的用户态就绪进程

1

如果PA在台上的时间足够长, $p_cpu \nearrow$, $p_pri \nearrow$

最近一次Swch中, p_pri 最小的进程PA (截止到 $t + \Delta t$, p_pri 最小, 优先级最高的进程) 抢到CPU



UNIX进程的调度状态



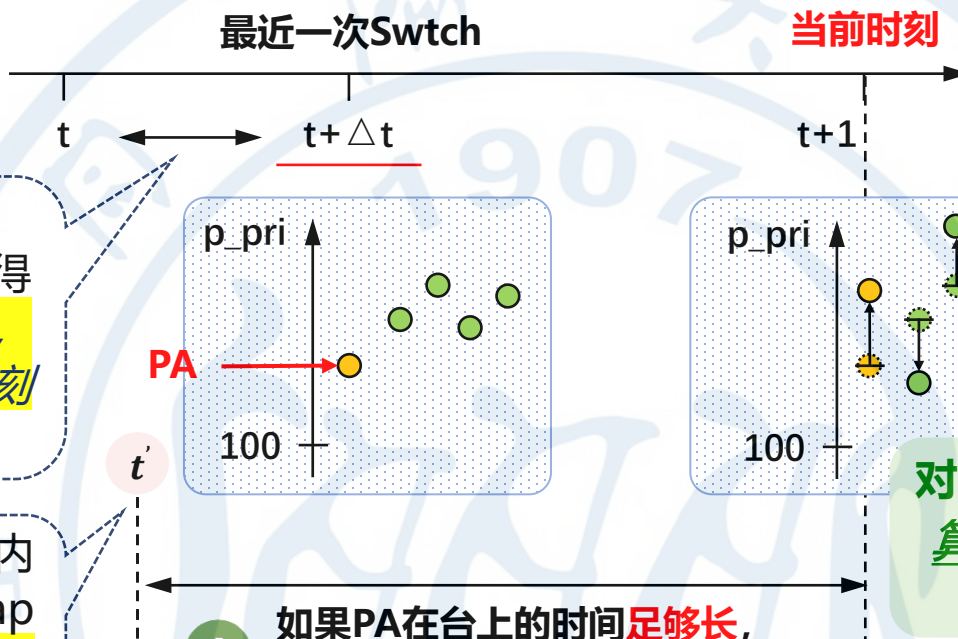
重算优先数

这些进程的优先数:

1. 在 t 时刻时钟中断中获得计算优先数 (此时, p_pri 体现截至到 t 时刻的 p_cpu) ;
2. 在 $t' \in [t, t + \Delta t]$ 时刻内曾在核心态下执行Trap末尾重算 (此时, p_pri 体现截至到 t' 的 p_cpu)

最近一次Swch

当前时刻



1

如果PA在台上的时间足够长,
 $p_cpu \nearrow$, $p_pri \nearrow$

1. 整数秒, 重算所有 $p_pri > PUSER$ 进程的优先数

既重算现运行进程, 也重算其他所有的用户态就绪进程

对现运行进程的重算满足:
算得的优先数 $>$ PA上台时的优先数

现运行进程优先级下降
设置RunRun

最近一次Swch中, p_pri 最小的进程PA (截止到 $t + \Delta t$, p_pri 最小, 优先级最高的进程) 抢到CPU



UNIX进程的调度状态



重算优先数

这些进程的优先数:

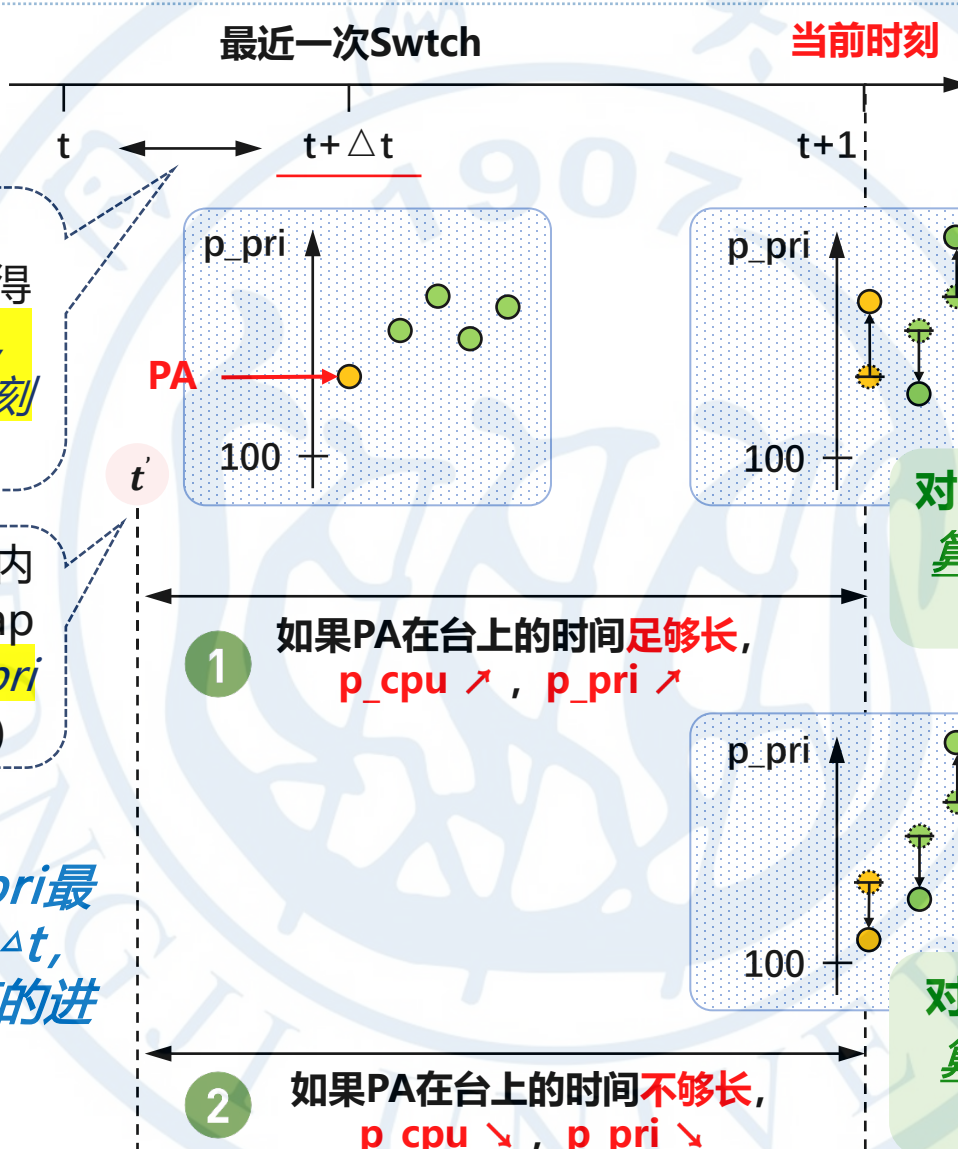
1. 在 t 时刻时钟中断中获得计算优先数 (此时, p_pri 体现截至到 t 时刻的 p_cpu) ;

2. 在 $t' \in [t, t + \Delta t]$ 时刻内曾在核心态下执行Trap末尾重算 (此时, p_pri 体现截至到 t' 的 p_cpu)

最近一次Swch中, p_pri 最小的进程PA (截止到 $t + \Delta t$, p_pri 最小, 优先级最高的进程) 抢到CPU

最近一次Swch

当前时刻



1. 整数秒, 重算所有 $p_pri > PUSER$ 进程的优先数

既重算现运行进程, 也重算其他所有的用户态就绪进程

对现运行进程的重算满足:
算得的优先数 $>$ PA上台时的优先数

现运行进程优先级下降
设置RunRun

对其他进程的重算满足:
算得的优先数 $>$ PA上台时的优先数

1秒计时到

设置RunRun



UNIX进程的调度状态



2. 整数秒，重算现运行进程的优先数

开始繁琐耗时的事务处理



1. 维护系统时间: `Time::lbolt -60`; 系统时间+1秒;
2. 开中断, 向中断控制器发送EOI指令;
3. 唤醒所有延时睡眠的进程;
4. 对所有进程 `p_time++`;
5. 所有进程 `p_cpu = max(0, p_cpu - SCHMAG)`;
6. 重算部分进程的优先数;
7. 如果RunIn被设置, 唤醒0#进程;
8. 现运行进程进行信号处理;
9. 重算当前进程的优先数。

可能花费较长时间
给现运行进程多一次重算优先数的机会

重算优先数



UNIX进程的调度状态



1. 重算优先数时，对RunRun的修改：



1. 整数秒，重算所有用户态就绪的进程的优先数



2. 整数秒，重算现运行进程的优先数



3. 系统调用末尾，重算现运行进程的优先数

Setpri函数完成进程优先数的计算

```
void Process::SetPri()
{
    int priority;
    ProcessManager& procMgr = Kernel::Instance().GetProcessManager();
    priority = this->p_cpu / 16;
    priority += ProcessManager::PUSER + this->p_nice;
    if ( priority > 255 )
    {
        priority = 255;
    }
    if ( priority > procMgr.CurPri )
    {
        procMgr.RunRun++;
    }
    this->p_pri = priority;
}
```

计算进程优先数

如果算得的优先数 > 现运行进程上台时的优先数

RunRun标志位被设置



重算优先数



UNIX进程的调度状态



1. 重算优先数时，对RunRun的修改：



1. 整数秒，重算所有用户态就绪的进程的优先数



2. 整数秒，重算现运行进程的优先数



3. 系统调用末尾，重算现运行进程的优先数

Setpri函数完成进程优先数的计算

计算进程优先数

$$p_pri = \min \{ 255, (p_cpu / 16 + PUSER + p_nice) \}$$

如果算得的优先数 > 现运行进程上台时的优先数

1秒计时到

现运行进程优先级下降

RunRun标志位被设置

现运行进程可能已经不适合继续在CPU上执行了

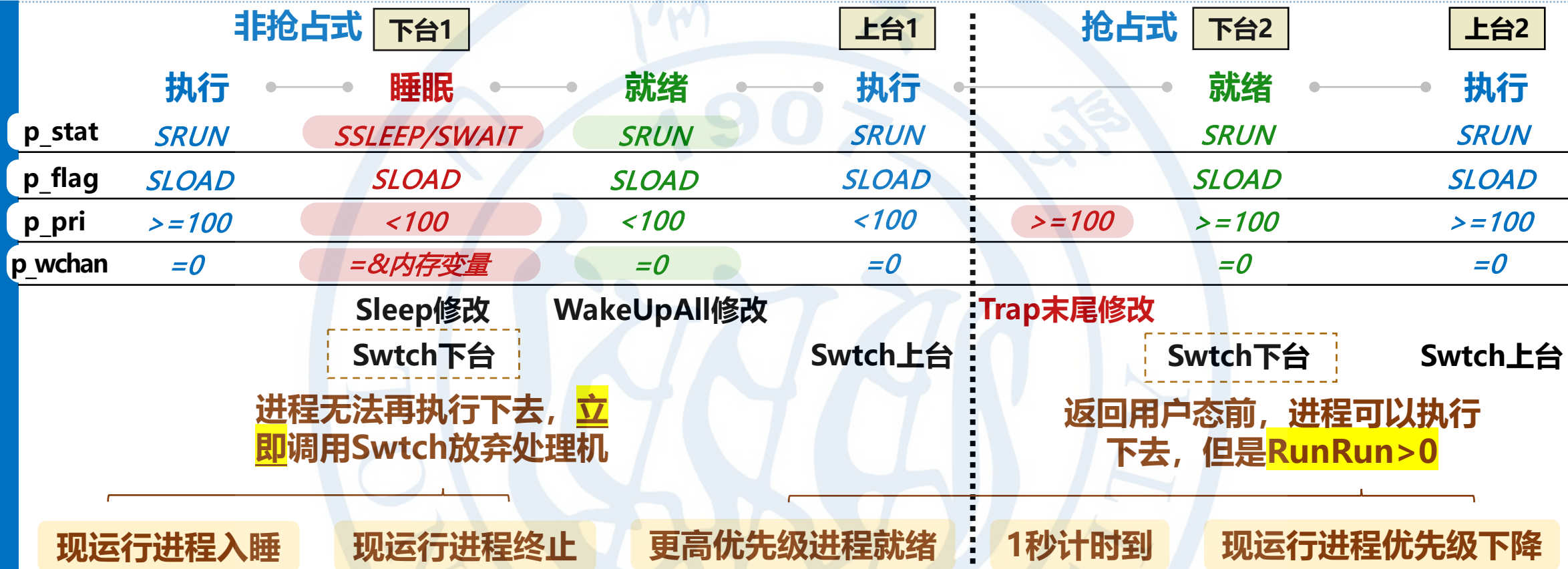
重算优先数



UNIX进程的调度状态

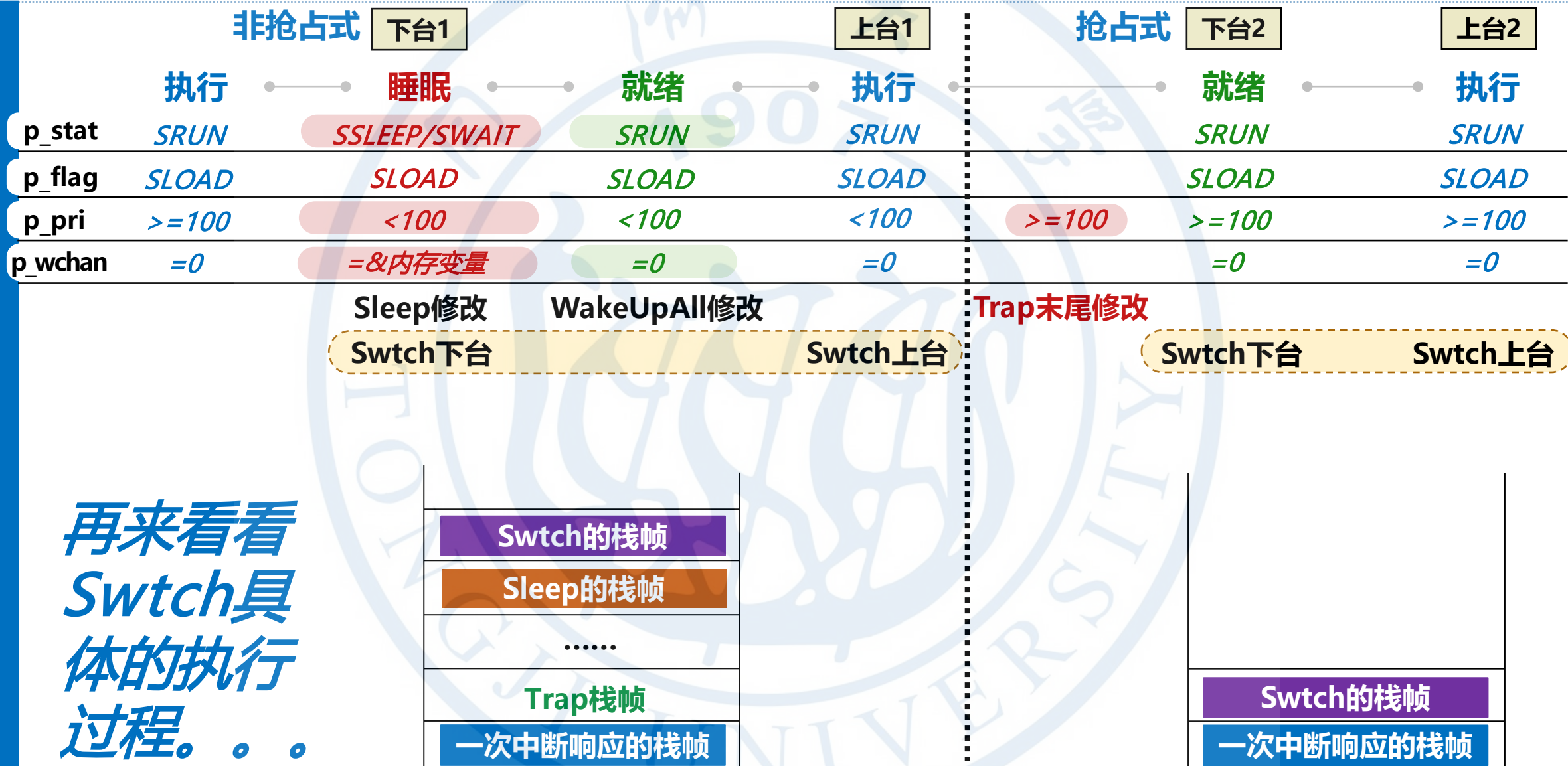


进程的调度状态





UNIX进程的上台与下台





UNIX进程的上台与下台



下台进程的现场保护

ProcessManager::Swth

保存现场

保存什么?

选新进程

恢复现场

return

非抢占式

$p_stat=SSLEEP/SWAIT, p_pri < 100,$
 $p_wchan \neq 0$

写入下台进程User结构
中的u_rsav[2]数组

esp
ebp

Swth的栈帧

Sleep的栈帧

.....

Trap栈帧

一次中断响应的栈帧

抢占式

$p_stat=SRUN, p_pri \geq 100$
 $p_wchan = 0$

写入下台进程User结构
中的u_rsav[2]数组

esp
ebp

Swth的栈帧

一次中断响应的栈帧



UNIX进程的上台与下台



新进程的选择

ProcessManager::Swch

保存现场

保存什么?

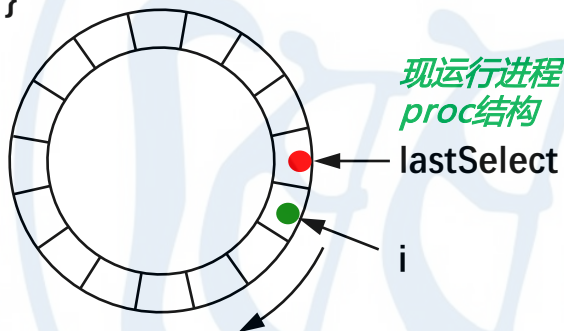
选新进程

1. 怎么选?

恢复现场

return

```
int priority = 256;
if ( process[i].p_pri < priority )
{
    best = i;
    priority = process[i].p_pri;
}
```



现运行进程
proc结构

lastSelect

i

从上一次被选中进程的下一个开始循环扫描，而不是每次从0#进程开始，保证各进程机会均等

e.g.

如果现运行进程是优先级并列高的进程，选择的结果?

select ()

清除RunRun标志, RunRun = 0

从上次查找到的进程位置开始线性循环扫描process数组，找图像在内存的优先级最高的就绪进程，即：满足条件：

$\text{Process::SRUN} == \text{process}[i].\text{p_stat} \ \&\& \ (\text{process}[i].\text{p_flag} \ \& \ \text{Process::SLOAD}) \neq 0$
的所有进程中，p_pri的值最小

找到?

N

执行hlt指令
等中断

Y
 $\text{this->CurPri} = \text{该进程的p_pri}$
记录该进程的在process数值中的下标为下一次查找的起始位置

返回指向该进程proc结构的指针
 $\text{return } \&\text{process}[\text{best}]$

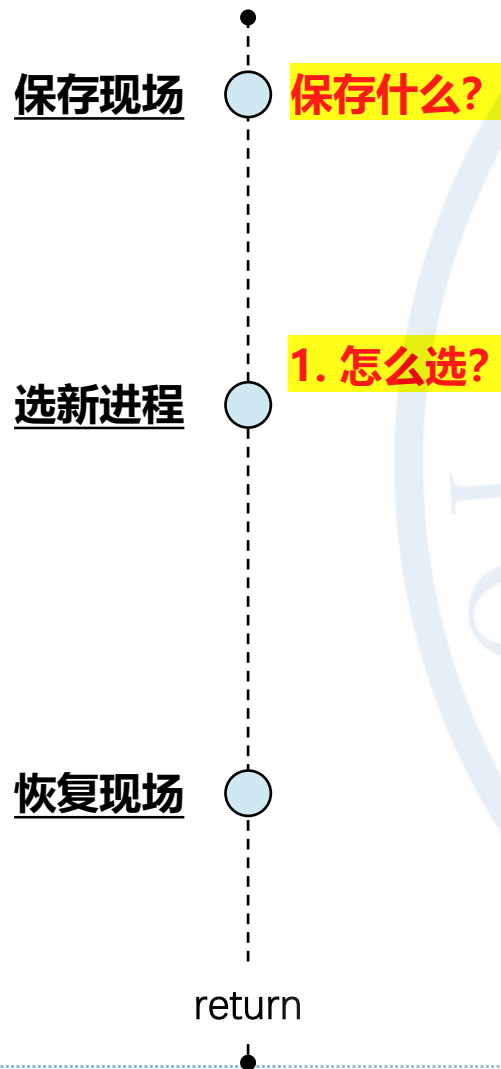


UNIX进程的上台与下台



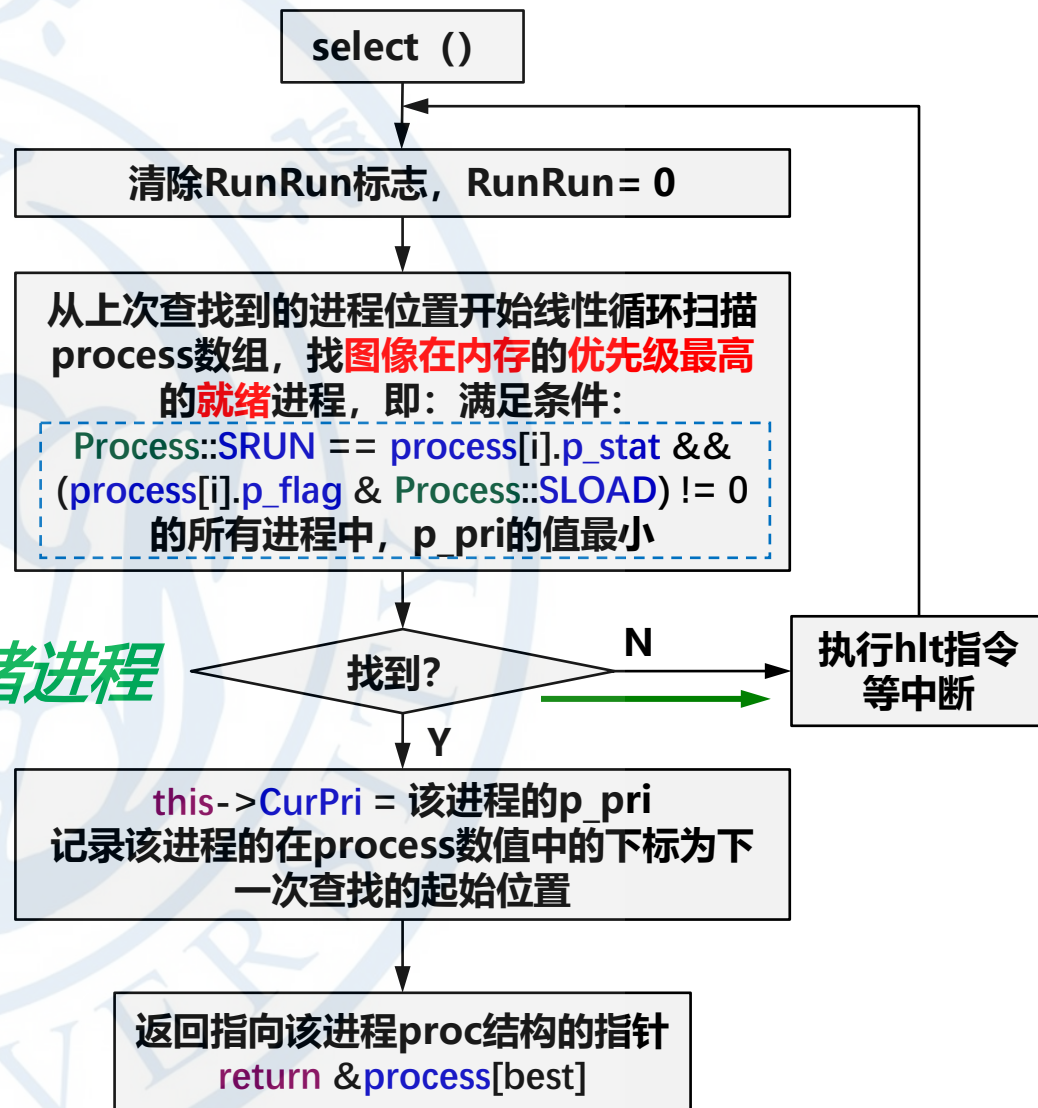
新进程的选择

ProcessManager::Swch



中断中会唤醒进程，
中断返回后，这里会
找到刚唤醒的进程

没有就绪进程





UNIX进程的上台与下台



新进程的选择

ProcessManager::Swrch

保存现场

保存什么?

重要的事要由重要的人来完成!!!

选新进程

1. 怎么选?

2. 谁来选?

0#

恢复现场

return

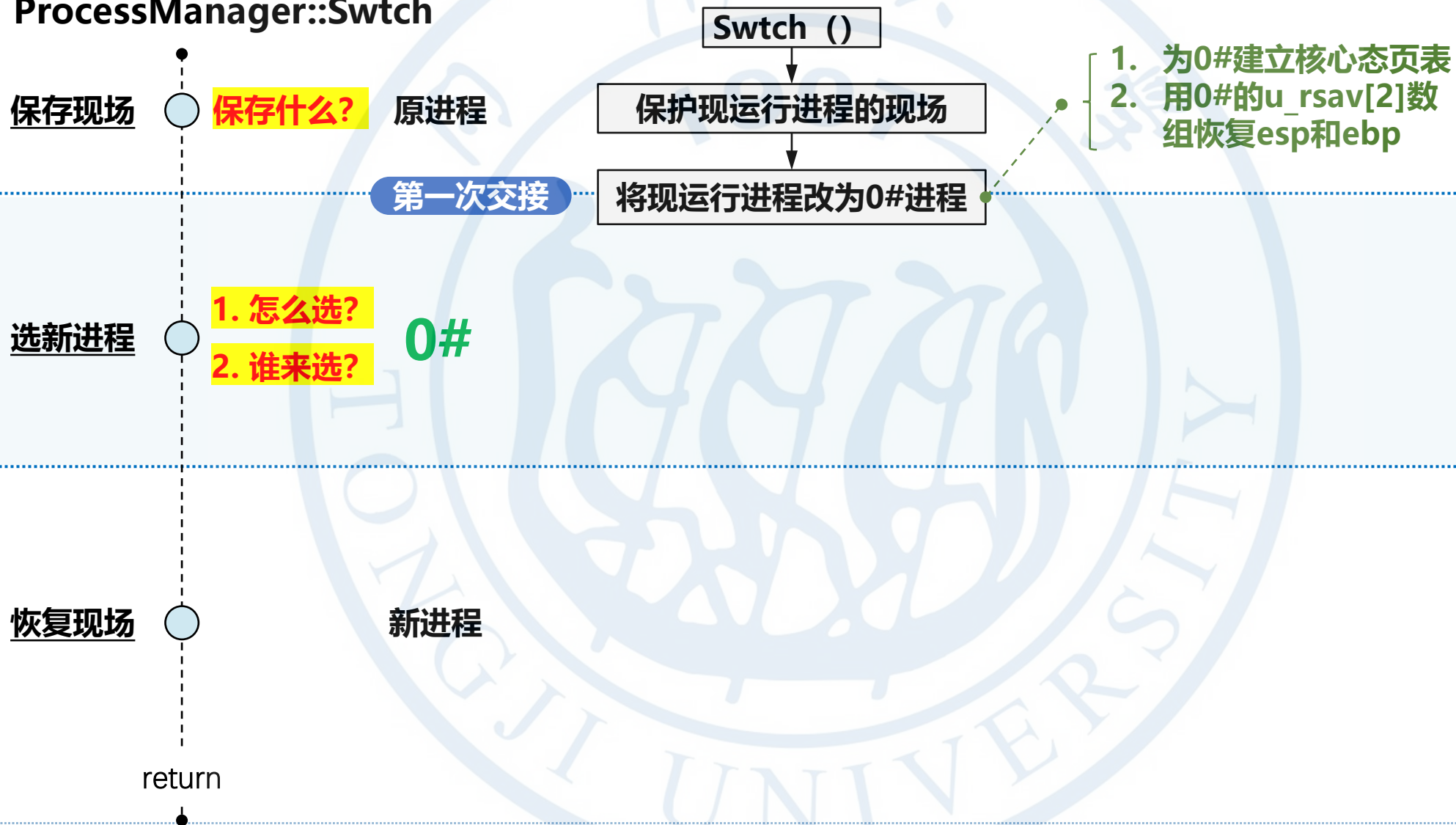


UNIX进程的上台与下台



新进程的选择

ProcessManager::Swth



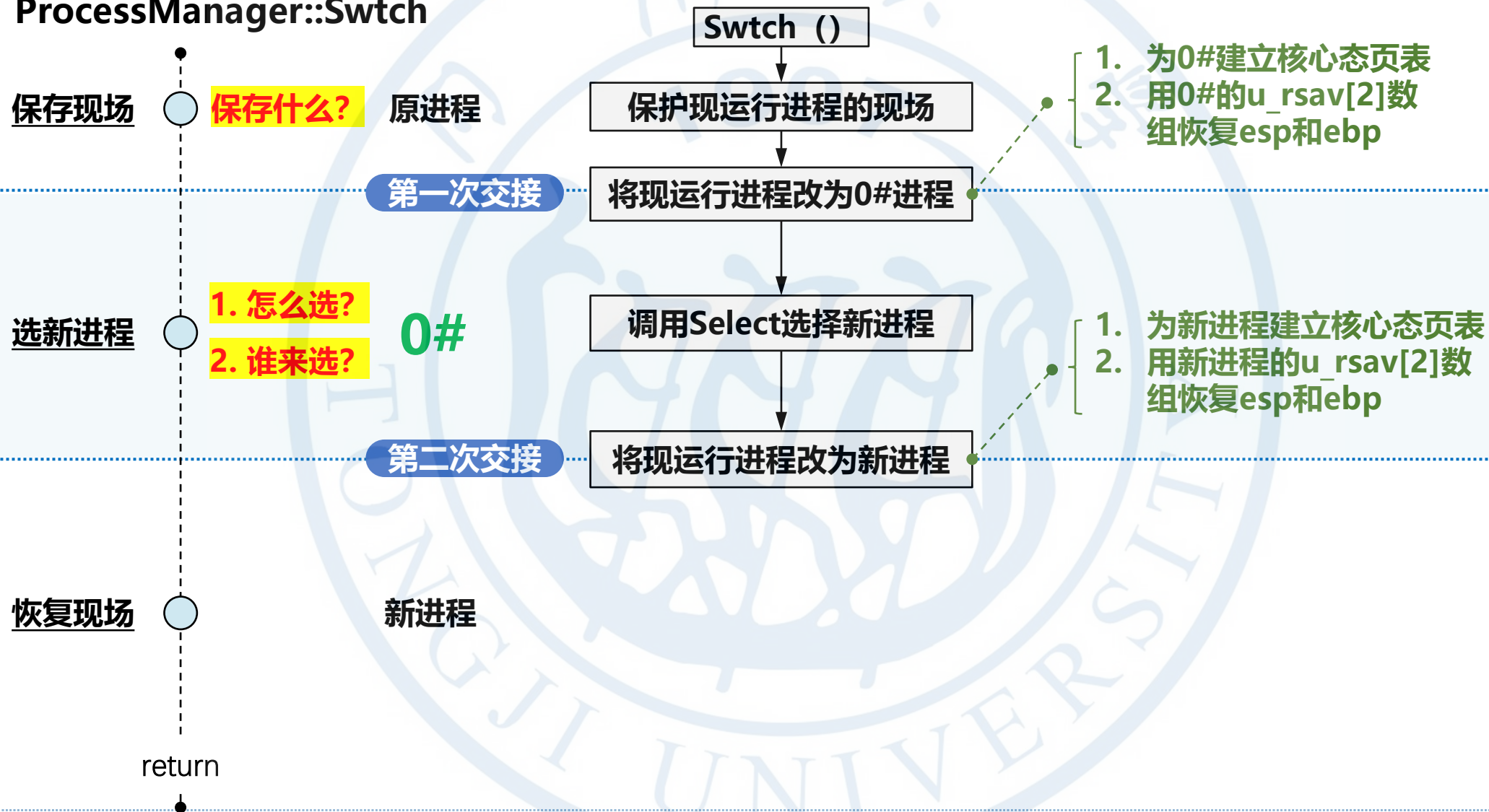


UNIX进程的上台与下台



新进程的选择

ProcessManager::Swch



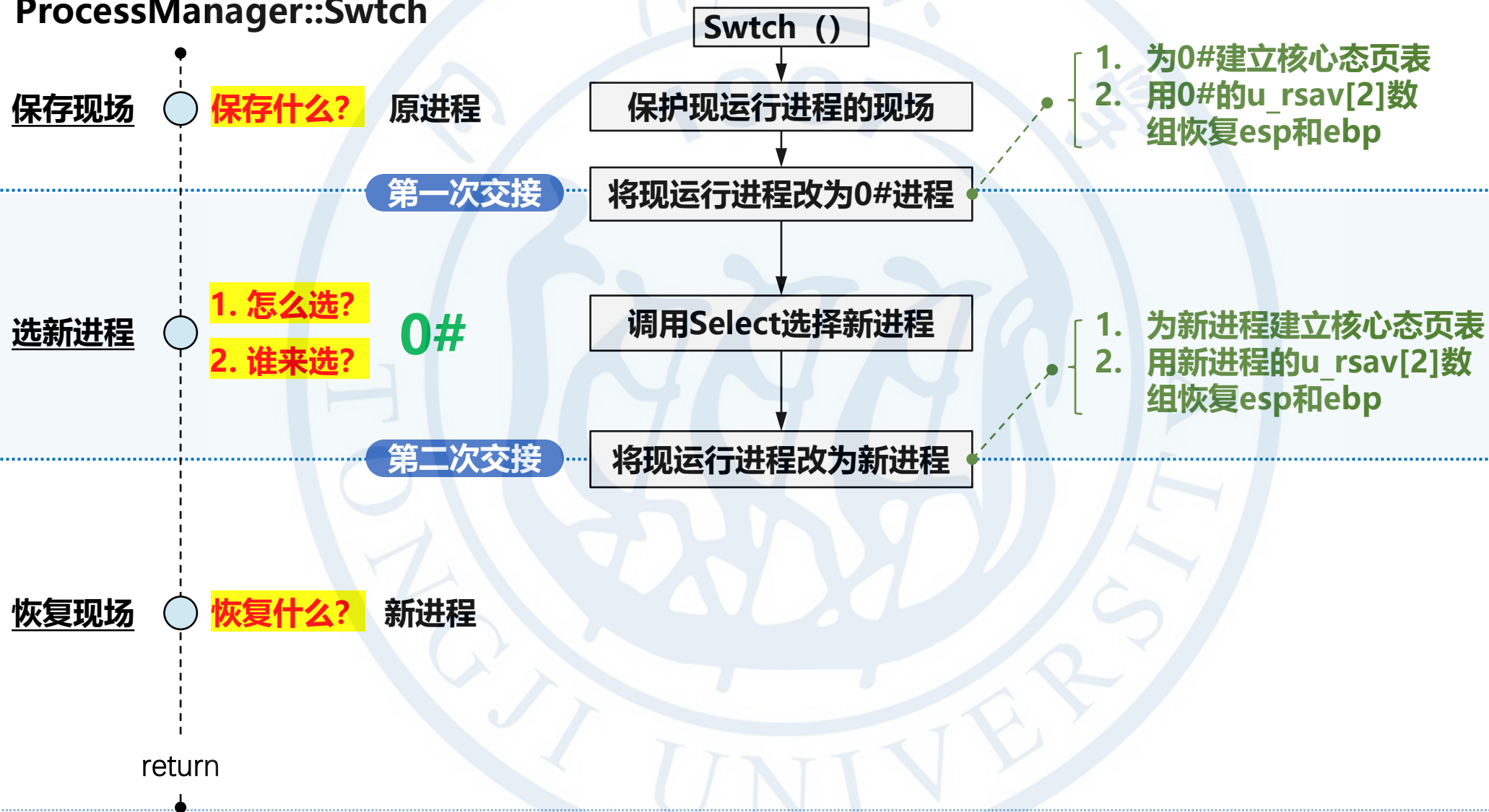


UNIX进程的上台与下台



上台进程的现场恢复

ProcessManager::Swth



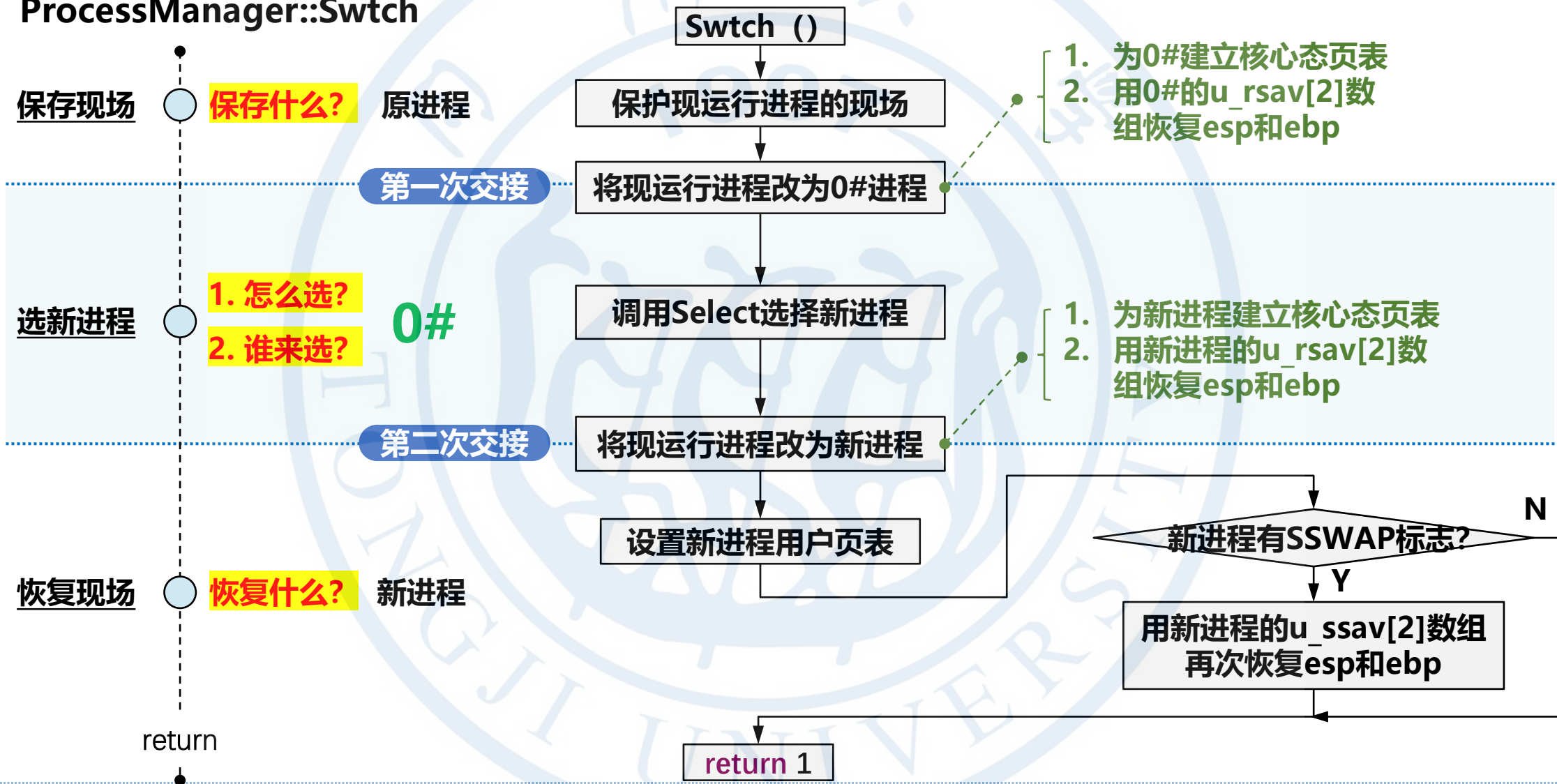


UNIX进程的上台与下台



上台进程的现场恢复

ProcessManager::Swch



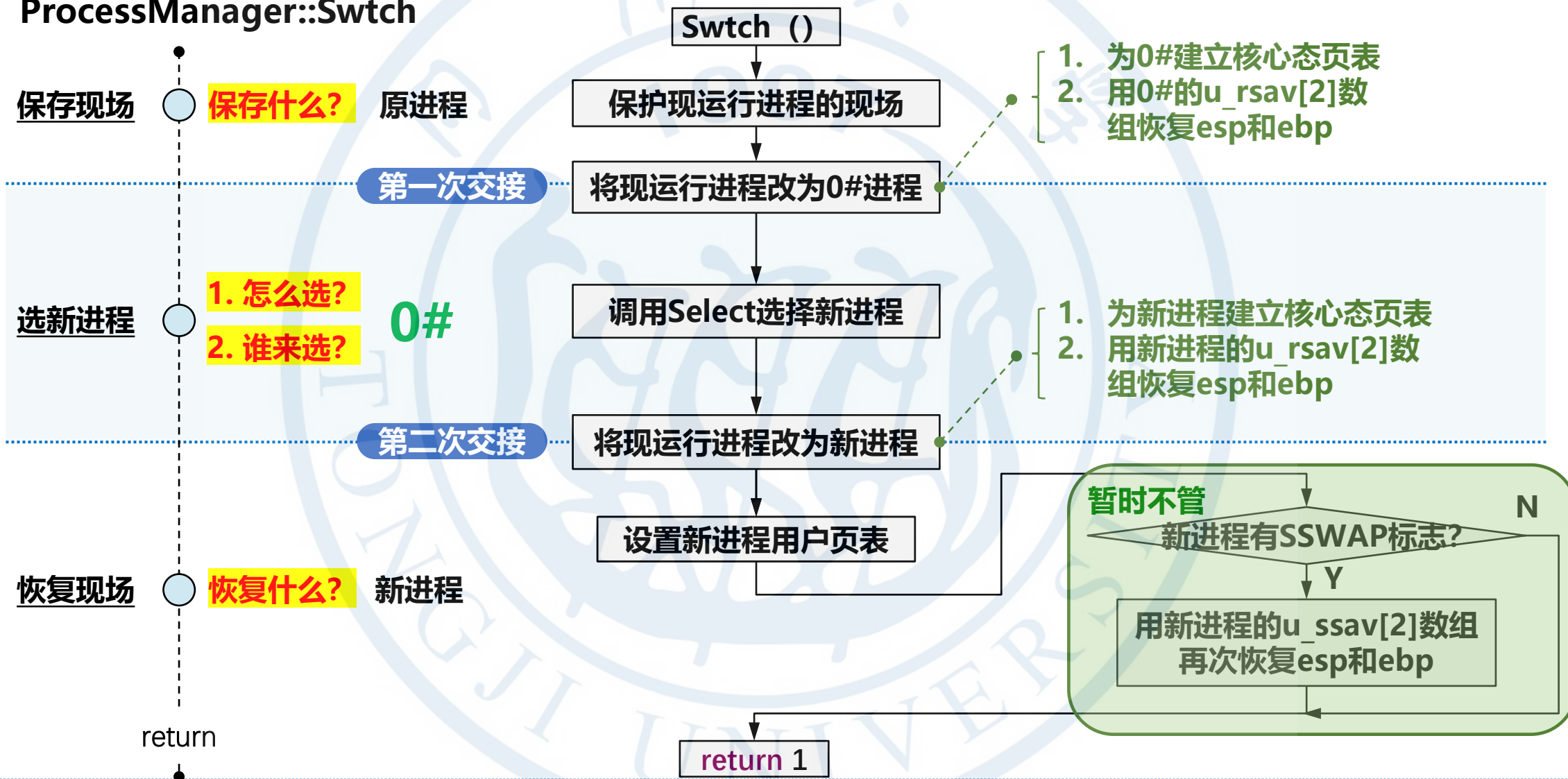


UNIX进程的上台与下台



上台进程的现场恢复

ProcessManager::Swch





UNIX进程的上台与下台



上台进程的现场恢复

ProcessManager::Swth

保存现场

保存什么?

原进程

非抢占式

$p_stat=SRUN$, $p_pri < 100$,
 $p_wchan = 0$

抢占式

$p_stat=SRUN$, $p_pri \geq 100$
 $p_wchan = 0$

选新进程

1. 怎么选?

2. 谁来选?

0#

第二次交接

建立页表, 用u_rsav[2]
数组中的值恢复

esp
ebp

Swth的栈帧

Sleep的栈帧

.....

Trap栈帧

一次中断响应的栈帧

恢复现场

恢复什么?

新进程

建立页表, 用u_rsav[2]
数组中的值恢复

esp
ebp

Swth的栈帧

一次中断响应的栈帧

return



UNIX进程的上台与下台



上台进程的现场恢复

ProcessManager::Swch

保存现场 ● **保存什么?** 原进程

选新进程 ● **1. 怎么选?**
2. 谁来选? 0#

恢复现场 ● **恢复什么?** 新进程

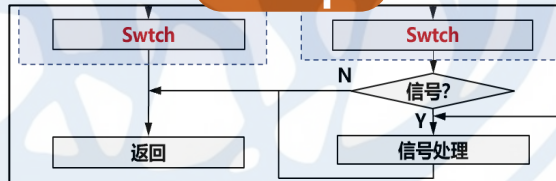
return **回哪里?**

非抢占式

$p_stat=SRUN$ $p_pri \geq 100$
 $p_wchan = 0$



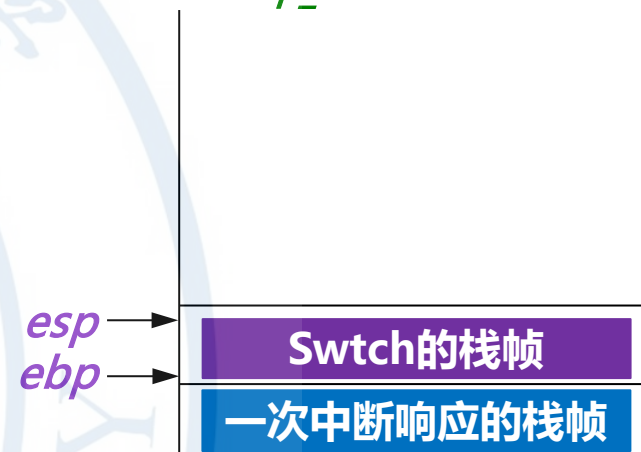
Sleep



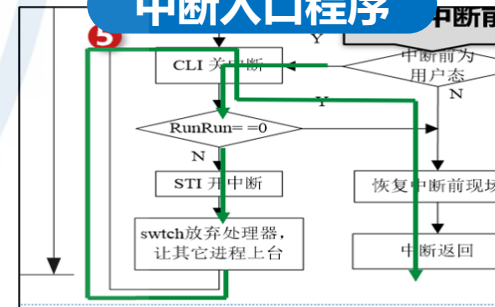
Trap

抢占式

$p_stat=SRUN, p_pri \geq 100$
 $p_wchan = 0$



中断入口程序



用户态



UNIX进程的上台与下台



关于进程切换的几个问题

ProcessManager::Swch

保存现场 ● **保存什么?** 原进程

第一次交接

选新进程 ●

1. 怎么选?

2. 谁来选?

0#

第二次交接

恢复现场 ●

恢复什么?

新进程

return

为什么三个进程可以合作完成一段代码?

三个进程经地址变换后
指向相同的物理地址

EIP →

ESP, EBP →

三个进程经地址变换后
指向不同的物理地址

Page Table 768# (0x201号页框)

	Page Base Address		s/u	r/w	p
0#	0		0	1	1
1#	1		0	1	1
.....					
1022#	1022		0	1	1
1023#	p_addr>>12		0	1	1

交接棒的过程中, 只改变了201#页框的最后一个PTE, EIP没变!!!



UNIX进程的上台与下台



关于进程切换的几个问题





UNIX进程的上台与下台



关于进程切换的几个问题

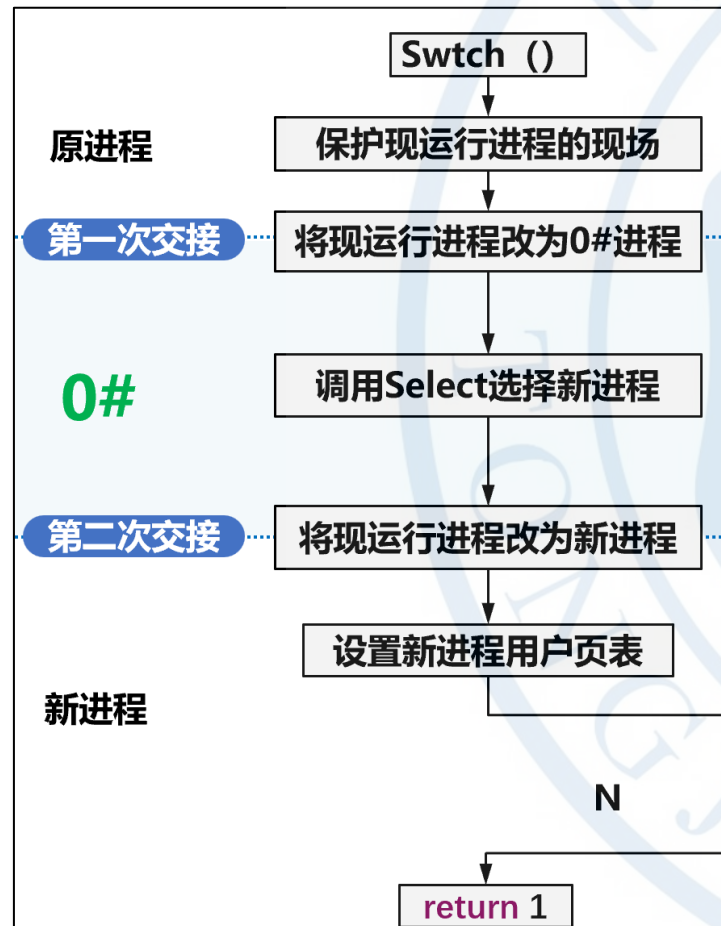
现运行进程入睡

更高优先级进程就绪

现运行进程优先级下降

现运行进程终止

1秒计时到



现运行进程已经无法继续在CPU上执行

Swch的第一棒和最后一棒一定是不同的进程

是时候该Swch了!!!

但是如果没有任何别的进程比我优先级高呢?

仍然最高

不是/并列最高

Swch后依然上台

Swch后下台

Swch的第一棒和最后一棒有可能是同一个进程



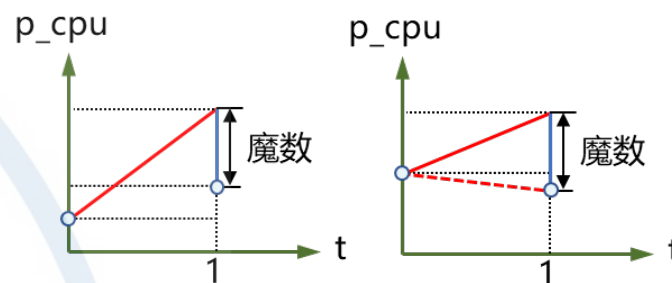
关于UNIX动态优先权调度算法



- ①每次时钟脉冲+1
- ②每秒 - 调度魔数

p_cpu

占用CPU足够多的进程 p_cpu ↗
占用CPU不够多的进程 p_cpu ↘



总结



关于UNIX动态优先权调度算法



- ①每次时钟脉冲+1
- ②每秒 - 调度魔数

p_cpu

占用CPU足够多的进程p_cpu ↗
占用CPU不够多的进程p_cpu ↘

计算

三次计算的时机

进程入睡

设置

p_pri

核心态就绪进程优先级高于用户态就绪进程

p_cpu ↗ p_pri ↗
p_cpu ↘ p_pri ↘

总结



关于UNIX动态优先权调度算法



总结

- ①每次时钟脉冲+1
- ②每秒 - 调度魔数

p_cpu

占用CPU足够多的进程p_cpu ↗
占用CPU不够多的进程p_cpu ↘

计算

三次计算的时机

进程入睡

设置

p_pri

核心态就绪进程优先级高于用户态就绪进程

p_cpu ↗ p_pri ↗
p_cpu ↘ p_pri ↘

重算进程优先数后，若重算的 p_pri > Curpri

高优先级进程就绪

设置RunRun

高优先级进程就绪

1秒计时到

现运行进程p_pri ↗
优先级 ↘



关于UNIX动态优先权调度算法



总结

- ①每次时钟脉冲+1
- ②每秒 - 调度魔数

p_cpu

占用CPU足够多的进程p_cpu ↗
占用CPU不够多的进程p_cpu ↘

计算

三次计算的时机

进程入睡

设置

p_pri

核心态就绪进程优先级高于用户态就绪进程

p_cpu ↗ p_pri ↗
p_cpu ↘ p_pri ↘

重算进程优先数后，若重算的 p_pri > Curpri

高优先级进程就绪

设置RunRun

高优先级进程就绪

1秒计时到

现运行进程p_pri ↗
优先级 ↘

中断/系统调用返回前

进程入睡/终止

立即

Swch被调用



UNIX的进程调度控制



非抢占式 下台1

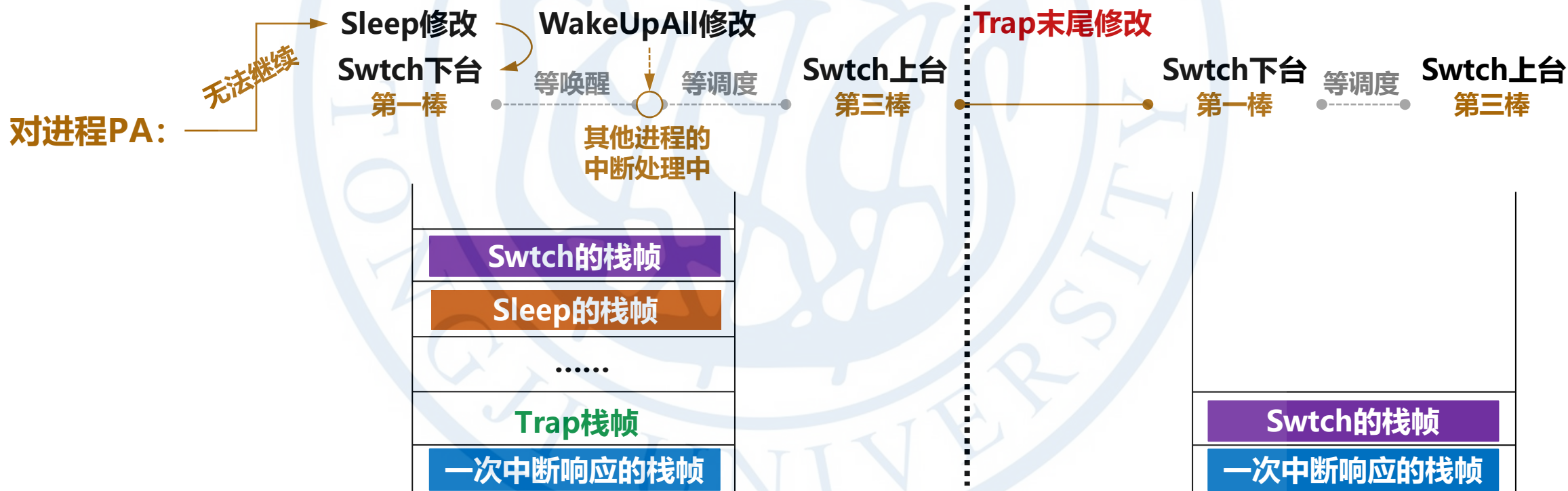
上台1

抢占式 下台2

上台2

	执行	睡眠	就绪	执行	就绪	执行
p_stat	SRUN	SSLEEP/SWAIT	SRUN	SRUN	SRUN	SRUN
p_flag	SLOAD	SLOAD	SLOAD	SLOAD	SLOAD	SLOAD
p_pri	≥ 100	< 100	< 100	< 100	≥ 100	≥ 100
p_wchan	=0	=&内存变量	=0	=0	=0	=0

总结





UNIX的进程调度控制



非抢占式

下台1

上台1

抢占式

下台2

上台2

	执行	睡眠	就绪	执行	就绪	执行
p_stat	SRUN	SSLEEP/SWAIT	SRUN	SRUN	SRUN	SRUN
p_flag	SLOAD	SLOAD	SLOAD	SLOAD	SLOAD	SLOAD
p_pri	≥ 100	< 100	< 100	< 100	≥ 100	≥ 100
p_wchan	=0	=&内存变量	=0	=0	=0	=0

Sleep修改

WakeUpAll修改

Trap末尾修改

Swch下台

Swch上台

Swch下台

Swch上台

进程无法再执行下去，**立即**调用Swch放弃处理机

返回用户态前，进程可以执行下去，但是**RunRun>0**

Swch的栈帧

Sleep的栈帧

.....

Trap栈帧

一次中断响应的栈帧

Swch的栈帧

一次中断响应的栈帧

总结



本节小结



1 UNIX进程的睡眠与唤醒过程

2 UNIX进程的切换调度

请阅读教材：171页 ~ 183页



E12：进程管理（UNIX的进程调度状态）